



@login_manager.user_loader

```
def load_user(user_id):  
    return User.query.get(int(user_id))
```

where does this go?

That function should live in the same place where you initialize `LoginManager`, after `login_manager = LoginManager(app)` and after your `User` model is defined/imported. [\[1\]](#) [\[2\]](#)

Typical structure

If you have a single `app.py`:

```
from flask import Flask  
from flask_sqlalchemy import SQLAlchemy  
from flask_login import LoginManager  
from models import User # or define User above  
  
app = Flask(__name__)  
db = SQLAlchemy(app)  
  
login_manager = LoginManager(app)  
login_manager.login_view = "login"  
  
@login_manager.user_loader  
def load_user(user_id):  
    return User.query.get(int(user_id))
```

- If you use a package structure (`__init__.py` plus `models.py`), put the `@login_manager.user_loader` near the app factory or in `__init__.py`, but always after `login_manager` and `User` are available. [\[3\]](#) [\[1\]](#)

✱✱

@app.route("/login", methods=["GET", "POST"])

```
def login():
```

```
    step = request.args.get("step", "email") # "email", "password", or "signup"  
    email = request.args.get("email", "").strip().lower()  
  
    if step == "email":
```

```

form = EmailOnlyLoginForm()
if form.validate_on_submit():
    email = form.email.data.strip().lower()
    user = User.query.filter_by(email=email).first()
    if user:
        return redirect(url_for("login", step="password", email=email))
    else:
        return redirect(url_for("login", step="signup", email=email))
return render_template("login.html", step="email", email=email, form=form)

if step == "password":
    form = PasswordOnlyLoginForm()
    if form.validate_on_submit():
        password = form.password.data
        user = User.query.filter_by(email=email).first()
        # TODO: check password properly (hash compare)
        if user and check_password_hash(user.password_hash, password): # placeholder!
            session["user_id"] = user.user_id # log the user in
            session["user_email"] = user.email
            return redirect(url_for("home"))
        else:
            form.password.errors.append("Falsches Passwort.")
    return render_template("login.html", step="password", email=email, form=form)

if step == "signup":
    form = SignupForm()
    if form.validate_on_submit():
        first_name = form.first_name.data
        last_name = form.last_name.data
        username = form.username.data
        password = form.password.data
        # TODO: create User row, hash password, commit
        user = User(
            email=email,
            password_hash=generate_password_hash(password),
            first_name=first_name,
            last_name=last_name,
            role="CUSTOMER"
        )
        db.session.add(user)
        db.session.commit()
        return redirect(url_for("home"))
    return render_template("login.html", step="signup", email=email, form=form)
else: # treat anything else as signup
    form = SignupForm()
    return render_template("login.html", step="signup", email=email, form=form)

```

do i replace all this with this short code?

```

from flask import request, redirect, url_for, render_template, flash
from flask_login import login_user

```

```

@app.route("/login", methods=["GET", "POST"])
def login():
    form = LoginForm()

```

```

if form.validate_on_submit():
    user = User.query.filter_by(email=form.email.data).first()
    if user and check_password_hash(user.password, form.password.data):
        login_user(user) # sets current_user and session for you
        return redirect(url_for("home"))
    else:
        flash("Ungültige E-Mail oder Passwort.")
        return render_template("login.html", form=form)

```

You do not have to throw away your multi-step flow; you only replace the parts where you manually set `session[...]` with `login_user(user)` and add Flask-Login on top. [\[4\]](#) [\[5\]](#)

1. Keep your existing multi-step logic

Your `step = "email" / "password" / "signup"` flow is fine and can stay as is; users like simple email-first flows. [\[5\]](#)

You only need to change the actual “log the user in” and later also the “after signup” part.

Updated password step with Flask-Login

```

from flask_login import login_user

if step == "password":
    form = PasswordOnlyLoginForm()
    if form.validate_on_submit():
        password = form.password.data
        user = User.query.filter_by(email=email).first()
        if user and check_password_hash(user.password_hash, password):
            login_user(user) # instead of session["user_id"] / session["user_email"]
            return redirect(url_for("home"))
        else:
            form.password.errors.append("Falsches Passwort.")
    return render_template("login.html", step="password", email=email, form=form)

```

Replace:

```

session["user_id"] = user.user_id
session["user_email"] = user.email

```

with `login_user(user)`. [\[6\]](#) [\[4\]](#)

Updated signup step (auto-login after registration)

```

if step == "signup":
    form = SignupForm()
    if form.validate_on_submit():
        first_name = form.first_name.data
        last_name = form.last_name.data
        username = form.username.data

```

```

password = form.password.data

user = User(
    email=email,
    password_hash=generate_password_hash(password),
    first_name=first_name,
    last_name=last_name,
    role="CUSTOMER"
)
db.session.add(user)
db.session.commit()

login_user(user) # log them in directly
return redirect(url_for("home"))

return render_template("login.html", step="signup", email=email, form=form)

```

So the overall route stays the same shape; only the session handling changes.^[5]

2. When would you use the “short” example?

The shorter example with a single `LoginForm` is just a simpler alternative flow (email + password on one form, no steps).^[5]

You would switch to it only if you want to change the UX to a classic one-page login; it is not required for using Flask-Login.

**

i executed pip install flask-login but the import doesnt work

The package name is `flask-login`, but the import name must be `flask_login` (underscore, not dash).^[7]

Check the import line

- Use this at the top of your file:

```
from flask_login import LoginManager, login_user, logout_user, login_required, current_user
```

- Do **not** write `import flask-login` (that will always fail in Python).

Quick troubleshooting steps

- In a terminal, confirm it is installed in the same environment as your app:

```
python -m pip show flask-login
```

If that shows nothing, install again **inside that environment**:

```
python -m pip install flask-login
```

✱

i need to activate the venv first

Yes, the virtualenv must be active before installing or importing packages so they land in the right environment. [\[8\]](#)

How to activate it

- On macOS/Linux (example name: venv):

```
source venv/bin/activate
```

- On Windows (CMD):

```
venv\Scripts\activate
```

Then run:

```
pip install flask-login
```

and start your Flask app from the same activated shell. [\[9\]](#)

✱

user = User.query.filter_by(email=email).first()

this doesnt work

`User.query.filter_by(email=email).first()` only works if your model and app are wired correctly; the failure usually comes from one of three things: wrong import, wrong field name, or no app/db context. [\[10\]](#)

1. Check your model and import

- Your `User` model must be the one bound to db:

```
from your_app import db
from your_app.models import User
```

- In the model, the column must be named exactly `email`:

```
email = db.Column(db.String(255), unique=True, nullable=False)
```

If the column is named differently (e.g. `user_email`), adjust the filter:

```
filter_by(user_email=email).
```

2. Ensure you are inside an app context

- In routes (inside `@app.route` functions) this is fine.
- In the Python shell or a script, you need:

```
from your_app import app
with app.app_context():
    user = User.query.filter_by(email=email).first()
```

3. Inspect the actual error message

- If you see `NameError: name 'User' is not defined` → import issue.
- If you see `AttributeError: type object 'User' has no attribute 'query'` → `User` is not a `db.Model` or `db` is different.
- If you see no error but `user` is `None` → there is simply no row with that email yet in the database.

If you paste the exact traceback, the failing line can be pinpointed more precisely.

✱✱

NoReferencedColumnError

`NoReferencedColumnError` means one of your foreign keys points to a column that does not exist (or is named differently) in the target table.^[11]

What is going wrong

- In SQLAlchemy, a foreign key like `db.ForeignKey("user.id")` expects a table `user` with a column `id` in the database schema.^[11]
- If your `User` model uses `user_id` as primary key (or another column name), but your foreign key still says `"user.id"`, SQLAlchemy raises `NoReferencedColumnError` because it cannot find a column named `id` on that table.^[11]

What to check in your code

- Compare your `User` model and all foreign keys that reference it, for example in `Order`:

```
class User(db.Model, UserMixin):
    __tablename__ = "user"
    user_id = db.Column(db.Integer, primary_key=True)
    # ...

class Order(db.Model):
    __tablename__ = "order"
```

```
# This will break:
user_id = db.Column(db.Integer, db.ForeignKey("user.id"))
```

- Fix by matching the actual column name:

```
user_id = db.Column(db.Integer, db.ForeignKey("user.user_id"))
```

or rename the primary key in `User` to `id` and adjust the DB migration. [\[11\]](#)

Once the foreign key target string exactly matches "`<table_name>.<column_name>`" of a real column, this error disappears.

✱

i change the primary key of User model from user_id to id

Changing the primary key from `user_id` to `id` is a good move and works well with Flask-Login's defaults. [\[24\]](#) [\[25\]](#)

What you need to adjust

- In the `User` model:

```
class User(db.Model, UserMixin):
    __tablename__ = "user"

    id = db.Column(db.Integer, primary_key=True) # was user_id
    # other fields unchanged
```

- In all other models' foreign keys, point to `"user.id"` now, for example in `Order`:

```
user_id = db.Column(db.Integer, db.ForeignKey("user.id"))
```

- In your code, replace usages of `user.user_id` with `user.id` and `session["user_id"] = user.user_id` (if still present) with `user.id` or, better, `login_user(user)` once Flask-Login is in. [\[26\]](#)

✱

from flask import Flask, redirect, render_template, request, session, url_for

```
from forms import EmailOnlyLoginForm, PasswordOnlyLoginForm, SignupForm, CartForm
from models import Order, User, db
from werkzeug.security import generate_password_hash, check_password_hash
from flask_login import LoginManager, login_user, login_required, logout_user, current_user
```

```

app = Flask(name)
login_manager = LoginManager(app)
login_manager.login_view = "login"

app.config["SECRET_KEY"] = "dev-secret-change-me"
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///farmersmarket.db"
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

db.init_app(app)

@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

@app.route("/")
def home():
    return render_template("home.html")

@app.route("/products")
def products():
    return render_template("products.html")

@app.route("/producers")
def producers():
    return render_template("producers.html")

@app.route("/maps")
def maps():
    return render_template("maps.html")

@app.route("/cart", methods=["GET", "POST"])
def cart():
    form = CartForm()

```

```

    user_logged_in = bool(session.get("user_id"))

    if request.method == "GET" and user_logged_in:
        form.email.data = session.get("user_email")
        form.first_name.data = session.get("first_name")
        form.last_name.data = session.get("last_name")
        form.address.data = session.get("address")
        form.city.data = session.get("city")
        form.zip_code.data = session.get("zip_code")

    if form.validate_on_submit():
        email = form.email.data
        first_name = form.first_name.data
        last_name = form.last_name.data
        address = form.address.data
        city = form.city.data

```



```
zip_code = form.zip_code.data
payment_method = form.payment_method.data
return render_template("cart.html", form=form, user_logged_in=user_logged_in)
```

```
@app.route("/login", methods=["GET", "POST"])
def login():
```

```
    step = request.args.get("step", "email") # "email", "password", or "signup"
    email = request.args.get("email", "").strip().lower()

    if step == "email":
        form = EmailOnlyLoginForm()
        if form.validate_on_submit():
            email = form.email.data.strip().lower()
            user = User.query.filter_by(email=email).first()
            if user:
                return redirect(url_for("login", step="password", email=email))
            else:
                return redirect(url_for("login", step="signup", email=email))
        return render_template("login.html", step="email", email=email, form=form)

    if step == "password":
        form = PasswordOnlyLoginForm()
        if form.validate_on_submit():
            password = form.password.data
            user = User.query.filter_by(email=email).first()
            if user and check_password_hash(user.password_hash, password): # placeholder!
                login_user(user)
                return redirect(url_for("home"))
            else:
                form.password.errors.append("Falsches Passwort.")
        return render_template("login.html", step="password", email=email, form=form)

    if step == "signup":
        form = SignupForm()
        if form.validate_on_submit():
            first_name = form.first_name.data
            last_name = form.last_name.data
            username = form.username.data
            password = form.password.data

            user = User(
                email=email,
                password_hash=generate_password_hash(password),
                first_name=first_name,
                last_name=last_name,
                role="CUSTOMER"
            )
            db.session.add(user)
            db.session.commit()
            login_user(user)
            return redirect(url_for("home"))
        return render_template("login.html", step="signup", email=email, form=form)
```

```

else: # treat anything else as signup
    form = SignupForm()
    return render_template("login.html", step="signup", email=email, form=form)

```

```

@app.route("/account")
@login_required
def account():
    user_orders = Order.query.filter_by(user_id=session["user_id"]).all()
    return render_template("account.html", user=current_user, orders=user_orders)

@app.route("/logout", methods=["POST"])
@login_required
def logout():
    logout_user()
    return redirect(url_for("home"))

@app.route("/business")
def business():
    return render_template("business.html")

if name == "main":
    app.run(debug=True)

```

this is my whole `app.py`, what do i change after implemeting flask login

With Flask-Login in place, you can stop using `session["user_id"]`/`session["user_email"]` and rely on `current_user` instead. Below is what to change, step by step. [\[28\]](#) [\[29\]](#)

1. Use `current_user` in `/cart`

Replace the manual session checks and prefill with `current_user.is_authenticated` and its fields.

```

from flask_login import current_user # already imported

@app.route("/cart", methods=["GET", "POST"])
def cart():
    form = CartForm()

    user_logged_in = current_user.is_authenticated # instead of session["user_id"]

    if request.method == "GET" and user_logged_in:
        form.email.data = current_user.email
        form.first_name.data = current_user.first_name
        form.last_name.data = current_user.last_name
        # only if you have these fields / from profile:
        # form.address.data = current_user.customer_profile.address
        # form.city.data = current_user.customer_profile.city
        # form.zip_code.data = current_user.customer_profile.zip_code

    if form.validate_on_submit():
        email = form.email.data
        first_name = form.first_name.data

```

```

        last_name = form.last_name.data
        address = form.address.data
        city = form.city.data
        zip_code = form.zip_code.data
        payment_method = form.payment_method.data
        # process order...

    return render_template("cart.html", form=form, user_logged_in=user_logged_in)

```

If you don't yet store address/city/zip on the user or profile, keep them as manual inputs for now.

2. Clean up /login route (logic is fine)

Your multi-step login is already correctly switched to `login_user(user)`, so you can remove session writes completely there (you already did).^[30]

Nothing else required in `/login` for Flask-Login.

3. Fix /account to use current_user

Do not read `session["user_id"]` anymore; use `current_user.id`, and you don't need to pass `user=current_user` if you don't want to.

```

@app.route("/account")
@login_required
def account():
    user_orders = Order.query.filter_by(user_id=current_user.id).all()
    return render_template("account.html", user=current_user, orders=user_orders)

```

In `account.html` you can use `{{ current_user.email }}` directly, but having `user=current_user` is also fine.

4. Remove unused session imports/usages

At the top you can drop `session` from the import if you no longer use it anywhere:

```

from flask import Flask, redirect, render_template, request, url_for # no session

```

If you still use `session` for other things (e.g. cart storage), keep it.

5. Verify LoginManager + user_loader

Your `LoginManager` setup and `@login_manager.user_loader` look correct as long as your `User` model now has `id` as primary key and inherits `UserMixin`.^{[29] [28]}

```

@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

```

After these changes, all auth-related checks should go through `current_user.is_authenticated`, and route protection uses `@login_required` instead of manual session checks.

✱

`user = User.query.filter_by(email=email).first()`

this line still fails, `NoReferencedColumnError`

`NoReferencedColumnError` is coming from your **model / foreign key definitions**, not from this query line itself; SQLAlchemy cannot find a column a foreign key points to. ^[31]

What is most likely wrong

After you renamed the primary key on `User` to `id`, you must also update every foreign key that references it, for example in `Order`:

- **Wrong (old):**

```
user_id = db.Column(db.Integer, db.ForeignKey("user.user_id"))
# or
user_id = db.Column(db.Integer, db.ForeignKey("user.id")) # while User still had user_id
```

- **Correct (with new PK):**

```
user_id = db.Column(db.Integer, db.ForeignKey("user.id"))
```

The string in `ForeignKey("user.id")` must match exactly `__tablename__ = "user"` and the column name `id` on `User`. ^[31]

If any foreign key still points to a non-existing column (e.g. `"user.user_id"` after you removed `user_id`), SQLAlchemy raises `NoReferencedColumnError` during model setup, and then any query (including `User.query.filter_by(...)`) appears to “fail” because the app never starts cleanly.

What to do now

1. Open all models that refer to `User` (e.g. `Order`, `CustomerProfile`, `ProducerProfile`) and fix their foreign keys to `"user.id"`.
2. Restart the app so the models are reloaded with the corrected foreign keys.
3. If you use migrations (Flask-Migrate/Alembic), generate and apply a migration for the PK/foreign-key change so the DB schema matches the models. ^[32] ^[33]

After the foreign keys are consistent, `user = User.query.filter_by(email=email).first()` will work as expected (it only returns `None` if there is simply no user with that email).

✱

logout user when tab is closed or programm is newly started

To log users out when the browser/tab is closed or the app restarts, you rely on **session cookies that are not permanent**, and you avoid "remember me". [\[34\]](#) [\[35\]](#)

1. Logout on browser/tab close

By default, Flask uses non-permanent sessions, which are deleted when the browser closes (unless the browser restores previous tabs). [\[36\]](#) [\[35\]](#)

- Ensure you do **not** make sessions permanent:

```
@app.before_request
def make_session_non_permanent():
    session.permanent = False # or just don't set it to True anywhere
```

- With Flask-Login:
 - Call `login_user(user, remember=False)` (the default) so no persistent "remember me" cookie is created. [\[35\]](#) [\[34\]](#)

Browsers with "restore previous session" enabled may still resurrect session cookies; you cannot fully control this from Flask. [\[37\]](#) [\[36\]](#)

2. Logout on program/server restart

When you restart the Flask app, session data can still be valid if:

- The `SECRET_KEY` stays the same, and
- The session cookie in the browser is still present. [\[38\]](#) [\[36\]](#)

If you really want all users logged out after restart, you can:

- Change `SECRET_KEY` on each restart (all existing cookies become invalid), or
- Implement your own token/version in the session and invalidate it when the app boots (more advanced pattern). [\[39\]](#) [\[36\]](#)

For your use case, the practical setup is:

- Use default Flask-Login behavior (`remember=False`, `session.permanent` left as `False`) so closing the browser logs users out in normal browsing settings.

✱

still that line fails,

```
user = User.query.filter_by(email=email).first()
```

this line still fails, `NoReferencedColumnError`

`NoReferencedColumnError` means SQLAlchemy is still unhappy with one of your **foreign keys**, not with this query line itself. Until the models are fixed, any query like `User.query...` will fail during startup.^[49]

1. What you should check now

Open all models that reference `User` and compare carefully:

```
class User(db.Model, UserMixin):
    __tablename__ = "user"
    id = db.Column(db.Integer, primary_key=True)
    # ...

class Order(db.Model):
    __tablename__ = "order"
    # check this:
    user_id = db.Column(db.Integer, db.ForeignKey("user.id"))
```

- `__tablename__` on `User` must be `"user"`.
- Primary key on `User` must be `id` (or you override `get_id()` appropriately).
- Every foreign key that points to `User` must use **exactly** `"user.id"` now:
 - In `Order`
 - In `CustomerProfile`
 - In `ProducerProfile`
 - Any other table that has a `user_id` with `ForeignKey(...)`.

If any one of them still says `"user.user_id"` or a wrong table/column name, you get `NoReferencedColumnError`.^[49]

2. Typical before/after fix

Example for `Order`:

```
# BEFORE (broken after renaming PK)
user_id = db.Column(db.Integer, db.ForeignKey("user.user_id"))

# AFTER (correct with PK = id)
user_id = db.Column(db.Integer, db.ForeignKey("user.id"))
```

Do the same for `CustomerProfile` / `ProducerProfile`:

```
user_id = db.Column(db.Integer, db.ForeignKey("user.id"))
user = db.relationship("User", back_populates="customer_profile")
```

3. Restart and (if needed) migrate

- After editing the models, stop the dev server and start it again so SQLAlchemy reloads the metadata.
- If you use Flask-Migrate/Alembic, generate and apply a migration to update the DB schema so foreign keys in the database also point to `user.id`.^[50] ^[51]

Only after all foreign keys are consistent will your line

```
user = User.query.filter_by(email=email).first()
```

run normally (returning a `User` or `None` instead of raising an exception).

✱

models.py

```
from datetime import datetime
from flask_sqlalchemy import SQLAlchemy
from flask_login import UserMixin
```

```
db = SQLAlchemy()
```

```
class User(db.Model, UserMixin):
```

```
    tablename = "user"
```

```
    id = db.Column(db.Integer, primary_key=True)
    email = db.Column(db.String(255), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)
    first_name = db.Column(db.String(100), nullable=False)
    last_name = db.Column(db.String(100), nullable=False)
    role = db.Column(db.String(20), nullable=False) # 'CUSTOMER' | 'PRODUCER'
    created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
```

```
    customer_profile = db.relationship("CustomerProfile", back_populates="user", uselist=False)
    producer_profile = db.relationship("ProducerProfile", back_populates="user", uselist=False)
```

```
class Address(db.Model):
```

```
    tablename = "address"
```

```
    address_id = db.Column(db.Integer, primary_key=True)
    street = db.Column(db.String(255), nullable=False)
    city = db.Column(db.String(100), nullable=False)
```

```
zip = db.Column(db.String(20), nullable=False)
country = db.Column(db.String(100), nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

customers = db.relationship("CustomerProfile", back_populates="address", lazy="dynamic")
producers = db.relationship("ProducerProfile", back_populates="address", lazy="dynamic")
```

class CustomerProfile(db.Model):

tablename = "customer_profile"

```
customer_id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True)
address_id = db.Column(db.Integer, db.ForeignKey("address.address_id"), nullable=True)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

user = db.relationship("User", back_populates="customer_profile")
address = db.relationship("Address", back_populates="customers")
```

class ProducerProfile(db.Model):

tablename = "producer_profile"

```
producer_id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True)
address_id = db.Column(db.Integer, db.ForeignKey("address.address_id"), nullable=True)
display_name = db.Column(db.String(255), nullable=False)
legal_name = db.Column(db.String(255), nullable=True)
tax_id = db.Column(db.String(50), nullable=True)
contact_email = db.Column(db.String(255), nullable=False)
contact_phone = db.Column(db.String(50), nullable=True)
verification_status = db.Column(db.String(20), nullable=False, default="pending")
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

user = db.relationship("User", back_populates="producer_profile")
products = db.relationship("Product", back_populates="producer", lazy="dynamic")
address = db.relationship("Address", back_populates="producers")
```

class Category(db.Model):

tablename = "category"

```
category_id = db.Column(db.Integer, primary_key=True)
name = db.Column(db.String(100), nullable=False, unique=True)

products = db.relationship("Product", back_populates="category", lazy="dynamic")
```

class Product(db.Model):

tablename = "product"


```

product_id = db.Column(db.Integer, primary_key=True)
producer_id = db.Column(db.Integer, db.ForeignKey("producer_profile.producer_id"), nullable=False)
category_id = db.Column(db.Integer, db.ForeignKey("category.category_id"), nullable=False)
name = db.Column(db.String(255), nullable=False)
description = db.Column(db.Text)
price = db.Column(db.Numeric(10, 2), nullable=False)
is_active = db.Column(db.Boolean, default=True, nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

producer = db.relationship("ProducerProfile", back_populates="products")
category = db.relationship("Category", back_populates="products")

cart_items = db.relationship("CartItem", back_populates="product", lazy="dynamic")
order_items = db.relationship("OrderItem", back_populates="product", lazy="dynamic")

```

class Cart(db.Model):

tablename = "cart"

```

cart_id = db.Column(db.Integer, primary_key=True)
customer_id = db.Column(db.Integer, db.ForeignKey("customer_profile.customer_id"), nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
updated_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

customer = db.relationship("CustomerProfile", backref=db.backref("carts", lazy="dynamic"))
items = db.relationship("CartItem", back_populates="cart", cascade="all, delete-orphan",

```

class CartItem(db.Model):

tablename = "cart_item"

```

cart_item_id = db.Column(db.Integer, primary_key=True)
cart_id = db.Column(db.Integer, db.ForeignKey("cart.cart_id"), nullable=False)
product_id = db.Column(db.Integer, db.ForeignKey("product.product_id"), nullable=False)
producer_id = db.Column(db.Integer, db.ForeignKey("producer_profile.producer_id"), nullable=False)
quantity = db.Column(db.Integer, nullable=False)

cart = db.relationship("Cart", back_populates="items")
product = db.relationship("Product", back_populates="cart_items")
producer = db.relationship("ProducerProfile")

```

class Order(db.Model):

tablename = "order"

```

order_id = db.Column(db.Integer, primary_key=True)
customer_id = db.Column(db.Integer, db.ForeignKey("customer_profile.customer_id"), nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
status = db.Column(db.String(20), nullable=False, default="pending")
total_amount = db.Column(db.Numeric(10, 2), nullable=False)

```

```
customer = db.relationship("CustomerProfile", backref=db.backref("orders", lazy="dynamic")
items = db.relationship("OrderItem", back_populates="order", cascade="all, delete-orphan")
```

```
class OrderItem(db.Model):
```

```
    tablename = "order_item"
```

```
    order_item_id = db.Column(db.Integer, primary_key=True)
    order_id = db.Column(db.Integer, db.ForeignKey("order.order_id"), nullable=False)
    product_id = db.Column(db.Integer, db.ForeignKey("product.product_id"), nullable=False)
    producer_id = db.Column(db.Integer, db.ForeignKey("producer_profile.producer_id"), nullable=False)
    quantity = db.Column(db.Integer, nullable=False)
    unit_price = db.Column(db.Numeric(10, 2), nullable=False)
    line_total = db.Column(db.Numeric(10, 2), nullable=False)

    order = db.relationship("Order", back_populates="items")
    product = db.relationship("Product", back_populates="order_items")
    producer = db.relationship("ProducerProfile")
```

did i miss anything?

The `models.py` file looks consistent; the `NoReferencedColumnError` is almost certainly coming from **outside** this file now, not from `User` or its FKs. [\[52\]](#)

1. Your foreign keys to `User` look correct

- `User`:

```
class User(db.Model, UserMixin):
    __tablename__ = "user"
    id = db.Column(db.Integer, primary_key=True)
    # ...
```

- References to `User`:

```
user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True)
```

in both `CustomerProfile` and `ProducerProfile`. That matches `__tablename__ = "user"` and column `id`, so this part is fine. [\[52\]](#)

2. Where to look next

`NoReferencedColumnError` is raised when SQLAlchemy builds the metadata and encounters a `ForeignKey("some_table.some_column")` where `some_column` does not exist on `some_table`. Since your models are clean, check: [\[52\]](#)

- Any **other model files** you might have (e.g. older copies, test models) still imported somewhere.

- Any **manual** ForeignKey **strings** in migrations (if using Alembic/Flask-Migrate) that reference old names like "user.user_id" or wrong table names.

Also check the **full error message**, it normally says something like:

```
sqlalchemy.exc.NoReferencedColumnError: Could not initialize target column for
ForeignKey 'user.user_id' on table 'xyz': table 'user' has no column named
'user_id' [52]
```

The quoted 'user.user_id' (or similar) tells you exactly which FK is wrong.

3. Quick steps to isolate the issue

- Temporarily comment out the **import of Order** in app.py and any code using it, then run the app.
- If the error disappears, the FK problem is in Order or OrderItem or their migrations.
- If it persists, read the exact FK name in the traceback and search your project for that string.

Once that remaining wrong ForeignKey("...") is corrected to match an existing __tablename__ and column, User.query.filter_by(email=email).first() will stop raising and behave normally.



OperationalError

```
sqlalchemy.exc.OperationalError: (sqlite3.OperationalError) no such column: user.id
[SQL: SELECT user.id AS user_id, user.email AS user_email, user.password_hash AS
user_password_hash, user.first_name AS user_first_name, user.last_name AS user_last_name,
user.role AS user_role, user.created_at AS user_created_at
FROM user
WHERE user.email = ?
LIMIT ? OFFSET ?]
[parameters: ('s_gecer24@stud.hwr-berlin.de', 1, 0)]
(Background on this error at: https://sqlalche.me/e/20/e3q8)
```

The code and models are now correct, but your **database still has the old schema** (with user_id instead of id), so SQLite says "no such column: user.id". [53]

What the error means

- SQLAlchemy sends this query: SELECT user.id AS user_id, ... FROM user WHERE user.email = ?.
- Your actual SQLite table user still has a column user_id, not id, because it was created before you changed the model.
- So SQLite raises no such column: user.id. [53]

Fix options (local dev, simplest first)

Because this is a dev project, the easiest fix is to **recreate the database** so it matches the new models:

1. Stop the Flask dev server.
2. Delete the SQLite file (careful: this removes all data):

```
rm farmersmarket.db      # macOS / Linux
# or delete in Explorer/Finder on Windows
```

3. In a Python shell or a small script with app context, recreate tables:

```
from app import app, db
with app.app_context():
    db.create_all()
```

or if you already do `db.create_all()` on startup, just run the app once. [\[54\]](#)

4. Try logging in/signing up again; the `user` table now has `id` and the query will work.

If you use migrations (Flask-Migrate)

Instead of deleting the DB, you can apply a migration that renames the column:

- Generate a migration after changing the model:

```
flask db migrate -m "Rename user_id to id on user"
flask db upgrade
```

- Inside the migration script, Alembic may need help with `op.alter_column` or a manual `op.execute` for SQLite. [\[55\]](#) [\[56\]](#)

For your current situation and speed, deleting `farmersmarket.db` and recreating tables is the most straightforward way to resolve this error.

✱

No such file or directory

The “No such file or directory” message means the path or filename you used does not exist at that location. [\[57\]](#)

Most likely in your case

You probably ran something like:

```
rm farmersmarket.db
```

in a folder where `farmersmarket.db` is not present, so the shell reports "No such file or directory".
[\[57\]](#)

How to fix it

- First, list files to see where the DB actually is:

```
ls
# or on Windows
dir
```

- If `farmersmarket.db` is in a subfolder (e.g. inside your project directory), `cd` into that folder first:

```
cd path/to/your/project
ls # verify farmersmarket.db is visible here
rm farmersmarket.db
```

- If the file has a different name or lives in a different path than `sqlite:///farmersmarket.db` (e.g. `instance/farmersmarket.db`), adjust the path accordingly before deleting.

✱

when i start the app, the navbar says my account, not login. when i press my account i get a login tho, so the functionality is correct just fix the titel

You need to switch the navbar condition from using the old session check to using `current_user.is_authenticated` so the label matches the new login state.
[\[58\]](#) [\[59\]](#)

What to change in base.html

Find the part in `base.html` where you show "Login" / "Mein Konto". It probably looks like this (old style):

```
{% if session["user_id"] %}
  <a href="{{ url_for('account') }}">Mein Konto</a>
{% else %}
  <a href="{{ url_for('login') }}">Login</a>
{% endif %}
```

Update it to use Flask-Login:

```
{% from "flask_login" import current_user %} {%# not needed if already injected #}

{% if current_user.is_authenticated %}
  <a href="{{ url_for('account') }}">Mein Konto</a>
{% else %}
```

```
<a href="{{ url_for('login') }}">Login</a>
{% endif %}
```

Flask-Login automatically injects `current_user` into templates, so you can safely rely on `current_user.is_authenticated` for the correct label. [\[60\]](#) [\[58\]](#)

✱

i deleted the db and recreated it. i guess the old structure remained, i get the same error

If the DB was recreated but the error still says `no such column: user.id`, either the old file is still used, or the `user` table was recreated with the old schema (still `user_id`). [\[61\]](#)

1. Confirm which DB file is used

- Check your config:

```
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///farmersmarket.db"
```

This means the file is created in the **current working directory of the process**, not necessarily next to `app.py`. [\[62\]](#)

- In the terminal where you run Flask:

```
pwd      # see current directory
ls       # confirm farmersmarket.db is here
```

If `farmersmarket.db` is not here, it was created elsewhere. Adjust `cd` and `rm` accordingly, then run `db.create_all()` again.

2. Inspect the actual table structure

Open the DB in the SQLite shell to see what columns the `user` table really has:

```
sqlite3 farmersmarket.db
.schema user
```

You should see:

```
CREATE TABLE user (
  id INTEGER NOT NULL,
  email VARCHAR(255) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  first_name VARCHAR(100) NOT NULL,
  last_name VARCHAR(100) NOT NULL,
  role VARCHAR(20) NOT NULL,
  created_at DATETIME NOT NULL,
```

```
PRIMARY KEY (id),  
UNIQUE (email)  
);
```

If you still see `user_id` instead of `id`, then:

- Either `models.py` used `user_id` when `db.create_all()` ran, or
- A migration recreated the old schema.

In that case:

1. Delete `farmersmarket.db` in the directory where you just inspected it.
2. Ensure `models.py` is exactly the version you posted (with `id` as primary key).
3. Start a Python shell in the same venv and directory:

```
python
```

Then:

```
from app import app, db  
with app.app_context():  
    db.create_all()
```

4. Recheck `.schema user` to confirm the column is now `id`.

3. Check for migrations overriding schema

If you use Flask-Migrate/Alembic and call `flask db upgrade` somewhere, it might be re-creating tables with the old column name from an old migration. [\[63\]](#) [\[64\]](#)

- Search your `migrations/versions/` for any `user` table creation with `user_id`.
- Either update those migrations or, during this refactor, temporarily avoid running migrations and rely only on `db.create_all()` until the schema is stable.

Once `.schema user` shows `id` and not `user_id`, the query

```
user = User.query.filter_by(email=email).first()
```

will stop raising `no such column: user.id` and just return a `User` or `None`.

✱

what is a shell, where do i execute this

A “shell” here just means a place where you can type commands or Python code directly: either your **system terminal** or the **interactive Python prompt**. [\[65\]](#)

1. System terminal (command line)

Use this for commands like `ls`, `rm`, `sqlite3`, `python`:

- On macOS/Linux: open **Terminal**.
- On Windows: open **Command Prompt** or **PowerShell**.

There you run, for example:

```
cd path/to/your/project
ls
sqlite3 farmersmarket.db
```

or:

```
cd path/to/your/project
python
```

2. Python shell (interactive Python)

This is what you get after running `python` in the terminal inside your venv:

```
cd path/to/your/project
source venv/bin/activate      # or venv\Scripts\activate on Windows
python
```

Now you see something like:

```
>>>
```

There you can paste:

```
from app import app, db
with app.app_context():
    db.create_all()
```

This runs inside your Flask app context and creates the tables in the DB configured in `app.config["SQLALCHEMY_DATABASE_URI"]`.^[65]



this is what i get

SQLite version 3.45.3 2024-04-15 13:34:05

Enter ".help" for usage hints.

sqlite>

That prompt means you are now **inside the SQLite shell**, connected to some database file. [\[66\]](#)

To debug your issue, you now need to:

1. Make sure it's the right DB file

If you just ran:

```
sqlite3 farmersmarket.db
```

in your project folder, then you are on the correct file. If you instead just typed `sqlite3` with no filename, you are in an in-memory DB. In that case, exit with: [\[66\]](#)

```
sqlite> .quit
```

and then run:

```
sqlite3 farmersmarket.db
```

in the same folder as your `farmersmarket.db`.

2. Inspect the user table

Inside the `sqlite>` prompt, run:

```
sqlite> .tables
```

Check that you see `user` in the list. Then:

```
sqlite> .schema user
```

Tell what you see there:

- If the schema shows `user_id INTEGER PRIMARY KEY`, the table is still using the old column name.
- If it shows `id INTEGER PRIMARY KEY`, the table matches your new model.

That output will show exactly why your query complains about `user.id`, and then the next step is either recreating the DB again or fixing the mismatch.



this is the table

```
CREATE TABLE user (  
id INTEGER NOT NULL,  
email VARCHAR(255) NOT NULL,  
password_hash VARCHAR(255) NOT NULL,  
first_name VARCHAR(100) NOT NULL,  
last_name VARCHAR(100) NOT NULL,  
role VARCHAR(20) NOT NULL,  
created_at DATETIME NOT NULL,  
PRIMARY KEY (id),  
UNIQUE (email)
```

The `user` table schema is now correct: it has `id` as primary key, so

`User.query.filter_by(email=email).first()` should no longer fail with “no such column: `user.id`”.
[\[67\]](#)

What to do next

- Exit SQLite:

```
sqlite> .quit
```

- Make sure your app is running in the **same folder** and `venv` where this `farmersmarket.db` lives:

```
cd path/to/your/project  
source venv/bin/activate # or venv\Scripts\activate on Windows  
python app.py
```

If you still see the same error in the Flask console, then Flask is using a **different** DB file/path than the one you just inspected. In that case:

- Print the URI in `app.py` to be sure:

```
print(app.config["SQLALCHEMY_DATABASE_URI"])
```

and confirm that this path matches the `farmersmarket.db` you opened with SQLite. If not, adjust the path or your working directory so both point to the same file. [\[68\]](#)

✱

the db file is in a folder, maybe thats the issue?

Yes, that can be the issue: Flask and SQLite must point to the **same path**, including folders. [\[69\]](#)

1. How Flask builds the path

You currently use:

```
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///farmersmarket.db"
```

This means: "use a file named `farmersmarket.db` in the current working directory where `flask run` / `python app.py` is executed". [\[69\]](#)

If your actual file is in e.g. `db/farmersmarket.db`, Flask will silently create another `farmersmarket.db` next to where you run the app, so you end up with two different DBs.

2. Point Flask to the correct folder

Option A: move the DB file next to `app.py` and keep the URI as is, then always `cd` into that folder before running the app.

Option B: keep the DB inside a folder (e.g. `db/`) and adjust the URI:

```
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///db/farmersmarket.db"
```

Then:

```
cd path/to/your/project
ls db    # should show farmersmarket.db
python app.py
```

Now the `user` table that you inspected with SQLite and the one SQLAlchemy uses are guaranteed to be the same file, and the `no such column: user.id` error should disappear.

✱

the farmersmarket.db is automatically created in instance folder

Flask is using the `instance` folder as the working directory for your SQLite file, so you need to either point SQLAlchemy there explicitly or always open that file when inspecting. [\[70\]](#)

Why it's in `instance/`

- When you run `flask run`, Flask uses the **instance folder** for relative SQLite paths like `sqlite:///farmersmarket.db`.^[70]
- That means your effective path is `instance/farmersmarket.db`, even if `app.py` lives one level above.

What to change

You have two clean options:

Option A: Keep using `instance/` (recommended)

Leave the URI as is:

```
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///farmersmarket.db"
```

But whenever you inspect or delete the DB, do it inside the `instance` folder:

```
cd path/to/your/project/instance
sqlite3 farmersmarket.db
.schema user
```

And if you need to delete:

```
rm farmersmarket.db
```

Then restart the app from the **project root** (where `flask run` or `python app.py` works).

Option B: Move DB out of `instance/`

If you prefer the file next to `app.py`, change the URI to an **absolute** or explicit relative path:

```
import os
basedir = os.path.abspath(os.path.dirname(__file__))
app.config["SQLALCHEMY_DATABASE_URI"] = f"sqlite:/// {os.path.join(basedir, 'farmersmarket.db')}
```

Now the DB is in the same folder as `app.py`, and you always inspect `farmersmarket.db` there.^[70]

Once Flask and your manual SQLite steps both point to the same file (the one whose `user` table already has `id`), the `no such column: user.id` error will be gone.



implement option A

You already almost have option A; only your workflow needs a small adjustment, not the code. [\[71\]](#)

1. Keep your current config

Leave `app.py` as it is:

```
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///farmersmarket.db"
```

This is correct for using `instance/farmersmarket.db` when running with `flask run`. [\[71\]](#)

2. Work with the right file

When you want to delete or inspect the DB, always go into the `instance` folder:

```
cd path/to/your/project/instance
ls                      # you should see farmersmarket.db
sqlite3 farmersmarket.db
.schema user           # to check the columns
```

To recreate the DB from Python:

```
cd path/to/your/project      # back to project root (where app.py is)
source venv/bin/activate     # or venv\Scripts\activate on Windows
python
```

Then in the Python shell:

```
from app import app, db
with app.app_context():
    db.create_all()
```

Now Flask and your manual SQLite steps both use `instance/farmersmarket.db`, so your `user` table with `id` will match the model and the previous error should disappear. [\[72\]](#) [\[71\]](#)

✱

how can i check the complete db structure

To see the complete DB structure, use the SQLite shell on your `farmersmarket.db`.

1. Open the correct database

From your project root:

```
cd instance
sqlite3 farmersmarket.db
```

Now you see the `sqlite>` prompt.^[73]

2. List all tables

```
sqlite> .tables
```

This shows every table (user, address, product, etc.).^[73]

3. Show schema for all tables

- Quick overview:

```
sqlite> .schema
```

- Or per table, e.g.:

```
sqlite> .schema user
sqlite> .schema customer_profile
sqlite> .schema order
```

This prints all `CREATE TABLE` statements, including columns and foreign keys.^[73]

When done, exit with:

```
sqlite> .quit
```

✱✱

how can i visually see the db

To see your SQLite DB visually, use a GUI like **DB Browser for SQLite** and open your `farmersmarket.db` file from there.^{[74] [75]}

1. Install a SQLite GUI

- Recommended: **DB Browser for SQLite** (free, works on Windows/macOS/Linux).^{[76] [74]}
- Download from: <https://sqlitebrowser.org> or via your package manager (e.g. Homebrew on macOS: `brew install db-browser-for-sqlite`).^{[74] [76]}

2. Open your farmersmarket.db

- Start DB Browser for SQLite.
- Click “Open Database” and navigate to your project’s `instance/farmersmarket.db` file.
- You will see:
 - A **Database Structure** tab listing tables and columns.
 - A **Browse Data** tab to view rows.
 - Foreign keys/relations via table definitions. [\[75\]](#) [\[74\]](#)

If you also want an ER diagram (boxes with lines between tables), you can additionally use tools like DbSchema or DbVisualizer that reverse-engineer an ERD from your SQLite file. [\[77\]](#) [\[78\]](#)

✱

what if the project is in github

If the project is on GitHub, you still inspect the database **locally**; GitHub only stores your code, not the running SQLite file. [\[89\]](#)

Where the DB lives with GitHub

- Your repo contains `app.py`, `models.py`, `templates`, etc. and maybe an empty `instance/` folder.
- When you run the app locally, SQLite creates `instance/farmersmarket.db` on **your machine**; this file usually is not committed to Git (often in `.gitignore`). [\[90\]](#) [\[89\]](#)

How to view it when using GitHub

- Clone or pull the project from GitHub as usual.
- Run the app locally so `farmersmarket.db` is created/updated in `instance/`.
- Then open that local `instance/farmersmarket.db` with DB Browser for SQLite or another GUI as described before; nothing changes because of GitHub. [\[91\]](#) [\[90\]](#)

If you want others to see the same data, you either share a dump or commit a copy of the `.db` file (not typical for production, but okay for demos or school projects).

✱

models.py

```
from datetime import datetime
from flask_sqlalchemy import SQLAlchemy
from flask_login import UserMixin

db = SQLAlchemy()
```

```
class User(db.Model, UserMixin):
```

```
tablename = "user"
```

```
id = db.Column(db.Integer, primary_key=True)
email = db.Column(db.String(255), unique=True, nullable=False)
password_hash = db.Column(db.String(255), nullable=False)
first_name = db.Column(db.String(100), nullable=False)
last_name = db.Column(db.String(100), nullable=False)
role = db.Column(db.String(20), nullable=False) # 'CUSTOMER' | 'PRODUCER'
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

customer_profile = db.relationship("CustomerProfile", back_populates="user", uselist=False)
producer_profile = db.relationship("ProducerProfile", back_populates="user", uselist=False)
```

```
class Address(db.Model):
```

```
tablename = "address"
```

```
address_id = db.Column(db.Integer, primary_key=True)
street = db.Column(db.String(255), nullable=False)
city = db.Column(db.String(100), nullable=False)
zip = db.Column(db.String(20), nullable=False)
country = db.Column(db.String(100), nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

customers = db.relationship("CustomerProfile", back_populates="address", lazy="dynamic")
producers = db.relationship("ProducerProfile", back_populates="address", lazy="dynamic")
```

```
class CustomerProfile(db.Model):
```

```
tablename = "customer_profile"
```

```
customer_id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True)
address_id = db.Column(db.Integer, db.ForeignKey("address.address_id"), nullable=True)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

user = db.relationship("User", back_populates="customer_profile")
address = db.relationship("Address", back_populates="customers")
```

```
class ProducerProfile(db.Model):
```

```
tablename = "producer_profile"
```

```
producer_id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True)
address_id = db.Column(db.Integer, db.ForeignKey("address.address_id"), nullable=True)
display_name = db.Column(db.String(255), nullable=False)
legal_name = db.Column(db.String(255), nullable=True)
tax_id = db.Column(db.String(50), nullable=True)
contact_email = db.Column(db.String(255), nullable=False)
```



```

contact_phone = db.Column(db.String(50), nullable=True)
verification_status = db.Column(db.String(20), nullable=False, default="pending")
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

user = db.relationship("User", back_populates="producer_profile")
products = db.relationship("Product", back_populates="producer", lazy="dynamic")
address = db.relationship("Address", back_populates="producers")

```

```

class Category(db.Model):
tablename = "category"

```

```

category_id = db.Column(db.Integer, primary_key=True)
name = db.Column(db.String(100), nullable=False, unique=True)

products = db.relationship("Product", back_populates="category", lazy="dynamic")

```

```

class Product(db.Model):
tablename = "product"

```

```

product_id = db.Column(db.Integer, primary_key=True)
producer_id = db.Column(db.Integer, db.ForeignKey("producer_profile.producer_id"), nullable=False)
category_id = db.Column(db.Integer, db.ForeignKey("category.category_id"), nullable=False)
name = db.Column(db.String(255), nullable=False)
description = db.Column(db.Text)
price = db.Column(db.Numeric(10, 2), nullable=False)
is_active = db.Column(db.Boolean, default=True, nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

producer = db.relationship("ProducerProfile", back_populates="products")
category = db.relationship("Category", back_populates="products")

cart_items = db.relationship("CartItem", back_populates="product", lazy="dynamic")
order_items = db.relationship("OrderItem", back_populates="product", lazy="dynamic")

```

```

class Cart(db.Model):
tablename = "cart"

```

```

cart_id = db.Column(db.Integer, primary_key=True)
customer_id = db.Column(db.Integer, db.ForeignKey("customer_profile.customer_id"), nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
updated_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

customer = db.relationship("CustomerProfile", backref=db.backref("carts", lazy="dynamic"))
items = db.relationship("CartItem", back_populates="cart", cascade="all, delete-orphan",

```

```
class CartItem(db.Model):
```

```
tablename = "cart_item"
```

```
cart_item_id = db.Column(db.Integer, primary_key=True)
cart_id = db.Column(db.Integer, db.ForeignKey("cart.cart_id"), nullable=False)
product_id = db.Column(db.Integer, db.ForeignKey("product.product_id"), nullable=False)
producer_id = db.Column(db.Integer, db.ForeignKey("producer_profile.producer_id"), nullable=False)
quantity = db.Column(db.Integer, nullable=False)

cart = db.relationship("Cart", back_populates="items")
product = db.relationship("Product", back_populates="cart_items")
producer = db.relationship("ProducerProfile")
```

```
class Order(db.Model):
```

```
tablename = "order"
```

```
order_id = db.Column(db.Integer, primary_key=True)
customer_id = db.Column(db.Integer, db.ForeignKey("customer_profile.customer_id"), nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)
status = db.Column(db.String(20), nullable=False, default="pending")
total_amount = db.Column(db.Numeric(10, 2), nullable=False)

customer = db.relationship("CustomerProfile", backref=db.backref("orders", lazy="dynamic"))
items = db.relationship("OrderItem", back_populates="order", cascade="all, delete-orphan")
```

```
class OrderItem(db.Model):
```

```
tablename = "order_item"
```

```
order_item_id = db.Column(db.Integer, primary_key=True)
order_id = db.Column(db.Integer, db.ForeignKey("order.order_id"), nullable=False)
product_id = db.Column(db.Integer, db.ForeignKey("product.product_id"), nullable=False)
producer_id = db.Column(db.Integer, db.ForeignKey("producer_profile.producer_id"), nullable=False)
quantity = db.Column(db.Integer, nullable=False)
unit_price = db.Column(db.Numeric(10, 2), nullable=False)
line_total = db.Column(db.Numeric(10, 2), nullable=False)

order = db.relationship("Order", back_populates="items")
product = db.relationship("Product", back_populates="order_items")
producer = db.relationship("ProducerProfile")
```

check if everything is fine,

```
from flask import Flask, redirect, render_template, request, url_for
```

```
from forms import EmailOnlyLoginForm, PasswordOnlyLoginForm, SignupForm, CartForm
```

```
from models import Order, User, db
```

```
from werkzeug.security import generate_password_hash, check_password_hash
```

```
from flask_login import LoginManager, login_user, login_required, logout_user, current_user
```

```

app = Flask(name)
login_manager = LoginManager(app)
login_manager.login_view = "login"

app.config["SECRET_KEY"] = "dev-secret-change-me"
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite:///farmersmarket.db"
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

db.init_app(app)

@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

@app.route("/")
def home():
    return render_template("home.html")

@app.route("/products")
def products():
    return render_template("products.html")

@app.route("/producers")
def producers():
    return render_template("producers.html")

@app.route("/maps")
def maps():
    return render_template("maps.html")

@app.route("/cart", methods=["GET", "POST"])
def cart():
    form = CartForm()

```

```

    user_logged_in = current_user.is_authenticated

    if request.method == "GET" and user_logged_in:
        form.email.data = current_user.email
        form.first_name.data = current_user.first_name
        form.last_name.data = current_user.last_name
        form.address.data = current_user.customer_profile.address
        form.city.data = current_user.customer_profile.city
        form.zip_code.data = current_user.customer_profile.zip_code.address.data = session.get("zip_code")

    if form.validate_on_submit():
        email = form.email.data
        first_name = form.first_name.data
        last_name = form.last_name.data
        address = form.address.data
        city = form.city.data

```

```
zip_code = form.zip_code.data
payment_method = form.payment_method.data
return render_template("cart.html", form=form, user_logged_in=user_logged_in)
```

```
@app.route("/login", methods=["GET", "POST"])
def login():
```

```
    step = request.args.get("step", "email") # "email", "password", or "signup"
    email = request.args.get("email", "").strip().lower()

    if step == "email":
        form = EmailOnlyLoginForm()
        if form.validate_on_submit():
            email = form.email.data.strip().lower()
            user = User.query.filter_by(email=email).first()
            if user:
                return redirect(url_for("login", step="password", email=email))
            else:
                return redirect(url_for("login", step="signup", email=email))
        return render_template("login.html", step="email", email=email, form=form)

    if step == "password":
        form = PasswordOnlyLoginForm()
        if form.validate_on_submit():
            password = form.password.data
            user = User.query.filter_by(email=email).first()
            if user and check_password_hash(user.password_hash, password): # placeholder!
                login_user(user)
                return redirect(url_for("home"))
            else:
                form.password.errors.append("Falsches Passwort.")
        return render_template("login.html", step="password", email=email, form=form)

    if step == "signup":
        form = SignupForm()
        if form.validate_on_submit():
            first_name = form.first_name.data
            last_name = form.last_name.data
            username = form.username.data
            password = form.password.data

            user = User(
                email=email,
                password_hash=generate_password_hash(password),
                first_name=first_name,
                last_name=last_name,
                role="CUSTOMER"
            )
            db.session.add(user)
            db.session.commit()
            login_user(user)
            return redirect(url_for("home"))
        return render_template("login.html", step="signup", email=email, form=form)
```

```
else: # treat anything else as signup
    form = SignupForm()
    return render_template("login.html", step="signup", email=email, form=form)
```

```
@app.route("/account")
```

```
@login_required
```

```
def account():
```

```
    user_orders = Order.query.filter_by(user_id=current_user.id).all()
```

```
    return render_template("account.html", user=current_user, orders=user_orders)
```

```
@app.route("/logout", methods=["POST"])
```

```
@login_required
```

```
def logout():
```

```
    logout_user()
```

```
    return redirect(url_for("home"))
```

```
@app.route("/business")
```

```
def business():
```

```
    return render_template("business.html")
```

```
if name == "main":
```

```
    app.run(debug=True)
```

```
from flask_wtf import FlaskForm
```

```
from wtforms import StringField, SelectField, PasswordField, SubmitField
```

```
from wtforms.validators import InputRequired, Email, Length, EqualTo, Regexp
```

```
class EmailOnlyLoginForm(FlaskForm):
```

```
    email = StringField(
```

```
        "E-Mail",
```

```
        validators=[InputRequired("Bitte E-Mail eingeben."), Email("Bitte gültige E-Mail eingeben.")]
```

```
    )
```

```
    submit = SubmitField("Weiter")
```

```
class PasswordOnlyLoginForm(FlaskForm):
```

```
    password = PasswordField(
```

```
        "Passwort",
```

```
        validators=[InputRequired("Bitte Passwort eingeben."), Length(min=6, message="Passwort  
muss mindestens 6 Zeichen lang sein.")]
```

```
    )
```

```
    submit = SubmitField("Login")
```

```
class SignupForm(FlaskForm):
```

```
    first_name = StringField(
```

```
        "Vorname", validators=[InputRequired(), Length(min=2, max=50)]
```

```
    )
```

```
    last_name = StringField(
```

```
        "Nachname", validators=[InputRequired(), Length(min=2, max=50)]
```

```
    )
```

```

username = StringField(
    "Benutzername",
    validators=[InputRequired("Bitte Benutzernamen eingeben."), Length(min=3, max=25,
    message="Benutzername muss zwischen 3 und 25 Zeichen lang sein.")])
)
password = PasswordField(
    "Passwort",
    validators=[InputRequired("Bitte Passwort eingeben."), Length(min=6, message="Passwort
muss mindestens 6 Zeichen lang sein.")])
)
password_confirm = PasswordField(
    "Passwort bestätigen",
    validators=[InputRequired("Bitte Passwort bestätigen."),
    EqualTo("password", message="Passwörter stimmen nicht überein.")],
)
submit = SubmitField("Jetzt beitreten")

```

```

class CartForm(FlaskForm):
    first_name = StringField("First name", validators=[InputRequired(), Length(min=2, max=50)])
    last_name = StringField("Last name", validators=[InputRequired(), Length(min=2, max=50)])
    address = StringField("Address", validators=[InputRequired(), Length(min=5, max=200)])
    city = StringField("City", validators=[InputRequired(), Length(min=2, max=50)])
    zip_code = StringField("ZIP code",
    validators=[
        InputRequired(),
        Length(min=4, max=10),
        Regexp(r" 92 +$", message="Bitte nur Ziffern eingeben.")
    ])
    email = StringField("Email", validators=[InputRequired(), Email()])
    payment_method = SelectField(
        "Payment method",
        choices=[
            ("pickup_cash", "Pay on pickup (cash)",
            ("digital_payment", "Digital payment"),
        ],
        validators=[InputRequired()],
    )
    submit = SubmitField("Place order")

```

Everything is almost consistent now; there are just a few concrete issues and simplifications to fix.

1. Models: structure looks correct

- User uses `id` as primary key and is referenced via `db.ForeignKey("user.id")` in both `CustomerProfile` and `ProducerProfile`, which matches your SQLite schema.^[93]
- All other foreign keys (`address.address_id`, `product.product_id`, etc.) match their respective primary keys, so there is no obvious `NoReferencedColumnError` source left in `models.py`.^[94]

You can keep `models.py` as it is.

2. /cart route: fix attribute access and a typo

`current_user.customer_profile.address` is a *relationship object*, not a string, so you must access its fields; also the last line is broken.

Change this block:

```
if request.method == "GET" and user_logged_in:
    form.email.data = current_user.email
    form.first_name.data = current_user.first_name
    form.last_name.data = current_user.last_name
    form.address.data = current_user.customer_profile.address
    form.city.data = current_user.customer_profile.city
    form.zip_code.data = current_user.customer_profile.zip_code.address.data = session.get("address")
```

to something safer, for now without profiles (until you actually create them):

```
if request.method == "GET" and user_logged_in:
    form.email.data = current_user.email
    form.first_name.data = current_user.first_name
    form.last_name.data = current_user.last_name
    # Leave address/city/zip empty for now, or fill once CustomerProfile/address rows exist
    # if current_user.customer_profile and current_user.customer_profile.address:
    #     form.address.data = current_user.customer_profile.address.street
    #     form.city.data = current_user.customer_profile.address.city
    #     form.zip_code.data = current_user.customer_profile.address.zip
```

- This avoids `AttributeError` when a user has no `CustomerProfile` yet and removes the broken `zip_code... = session.get("address")` part.
- Once you start creating `CustomerProfile` + `Address` rows on signup or later, uncomment and use the nested attributes.

The rest of the `/cart` route is fine for now.

3. /account route: adapt to current schema

Your `Order` model does not have `user_id`, it has `customer_id` pointing to `CustomerProfile.customer_id`. So this line will not work:^[93]

```
user_orders = Order.query.filter_by(user_id=current_user.id).all()
```

You have two options:

A. Quick, schema-agnostic placeholder

For now, if you haven't wired carts/orders to profiles yet, you can simply show no orders or all orders as a placeholder:

```
@app.route("/account")
@login_required
def account():
    user_orders = [] # or Order.query.all() as a temporary view
    return render_template("account.html", user=current_user, orders=user_orders)
```

B. Correct linkage via CustomerProfile (later)

Once you create a CustomerProfile for each user and link orders to it, you can do:

```
@app.route("/account")
@login_required
def account():
    if current_user.customer_profile:
        user_orders = current_user.customer_profile.orders.all()
    else:
        user_orders = []
    return render_template("account.html", user=current_user, orders=user_orders)
```

This uses the `orders` backref defined on `CustomerProfile` and stays consistent with your `Order.customer_id` design.

4. /login route: now correct with Flask-Login

- The multi-step logic with `step="email" / "password" / "signup"` is compatible with Flask-Login; using `login_user(user)` in password and signup branches is correct. [\[95\]](#)
- `User.query.filter_by(email=email).first()` now matches the `user` table schema (`email` column exists), so it will work as long as you are pointing to the correct `farmersmarket.db` in `instance/`. [\[93\]](#)

No structural changes needed there.

5. Small cleanups and checks

- Your imports at the top of `app.py` no longer need `session`; you can keep the import slim:

```
from flask import Flask, redirect, render_template, request, url_for
```

- Make sure your navbar uses `current_user.is_authenticated` to switch between "Login" and "Mein Konto", not `session["user_id"]`, so the label is correct with Flask-Login. [\[96\]](#) [\[97\]](#)

Example in `base.html`:


```
{% if current_user.is_authenticated %}
  <a href="{{ url_for('account') }}">Mein Konto</a>
{% else %}
  <a href="{{ url_for('login') }}">Login</a>
{% endif %}
```

If you apply these adjustments (especially the `/cart` prefill block and `/account` query), the code structure matches your DB schema and Flask-Login setup cleanly.

**

user_orders = Order.query.filter_by(user_id=current_user.id).all()

```
return render_template("account.html", user=current_user, orders=user_orders)
```

`Order` has no `user_id` column in your models, so this line will always be wrong for your current schema.^[98]

Correct pattern for your models

Your `Order` model:

```
class Order(db.Model):
    __tablename__ = "order"

    order_id = db.Column(db.Integer, primary_key=True)
    customer_id = db.Column(db.Integer, db.ForeignKey("customer_profile.customer_id"), nullable=True)
    # ...
```

So orders belong to a **CustomerProfile**, not directly to `User`.^[98]

Use this pattern instead:

```
@app.route("/account")
@login_required
def account():
    if current_user.customer_profile:
        user_orders = current_user.customer_profile.orders.all()
    else:
        user_orders = []
    return render_template("account.html", user=current_user, orders=user_orders)
```

- `current_user.customer_profile` comes from the relationship on `User`.
- `.orders` is the backref on `CustomerProfile` you defined in `Order` (`customer = db.relationship("CustomerProfile", backref=db.backref("orders", lazy="dynamic"))`).^[98]

✱

sqlalchemy.exc.OperationalError:
(sqlite3.OperationalError) no such table: user
[SQL: SELECT user.id AS user_id, user.email AS
user_email, user.password_hash AS
user_password_hash, user.first_name AS
user_first_name, user.last_name AS
user_last_name, user.role AS user_role,
user.created_at AS user_created_at FROM user
WHERE user.email = ? LIMIT ? OFFSET ?]
[parameters: ('s_gecer24@stud.hwr-berlin.de', 1,
0)] (Background on this error at:
<https://sqlalche.me/e/20/e3q8>)

still the same error on the same line, somehow it cant find the column

The error now says **“no such table: user”**, not **“no such column: user.id”**. That means the `user` table does not exist in the SQLite file Flask is actually using. ^[99]

What this tells you

- SQLAlchemy is executing:
`FROM user WHERE user.email = ?`
- SQLite replies: there is **no table named** `user` in that database file. ^[99]
- In your `instance/farmersmarket.db` that you inspected earlier, the table exists, so Flask must be pointing to a **different** database file, or `db.create_all()` was never run for this file.

Concrete steps to fix it

1. Confirm which DB file Flask uses

In `app.py`, temporarily add a print near the top:

```
print("DB URI:", app.config["SQLALCHEMY_DATABASE_URI"])
```

Start the app from your project root:

```
cd path/to/your/project
source venv/bin/activate
flask run    # or python app.py
```

- If it prints `sqlite:///farmersmarket.db`, with `flask run` this means:
`instance/farmersmarket.db`.^[100]

2. Create tables in that exact DB

From the same directory where you run the app:

```
python
```

Then in the Python shell:

```
from app import app, db
with app.app_context():
    db.create_all()
```

This will create the `user` (and other) tables in the DB referenced by `SQLALCHEMY_DATABASE_URI`.^[100]

3. Verify the tables exist in that file

Still in the same project root, open the DB that Flask uses:

```
cd instance          # if URI is sqlite:///farmersmarket.db
sqlite3 farmersmarket.db
.tables
.schema user
```

- `.tables` must list `user`.
- `.schema user` must show `CREATE TABLE user (...)`.

If `.tables` shows nothing, you ran `db.create_all()` in a different folder or with a different URI.

4. Make your workflow consistent

- Always `cd` into the project root before:
 - Running `flask run` / `python app.py`.
 - Running `python` for `db.create_all()`.
- Always open `instance/farmersmarket.db` for inspection (for `sqlite:///farmersmarket.db` with `flask run`).^[99] ^[100]

Once the **user** table exists in the same database file that the app uses, the query line

```
user = User.query.filter_by(email=email).first()
```

will stop raising `No such table: user` and simply return a user or `None`.

✱

how do i get one level up in the directory

Use `cd ..` to go one folder up.

- macOS/Linux/Windows (CMD/PowerShell):

```
cd ..
```

This moves you from e.g. `.../project/instance` back to `.../project`.

return render_template("account.html", user=current_user, orders=customer_profile)

why does this not render?

`customer_profile` is not a list of orders, so passing it as `orders=customer_profile` will not render an order list and may break any template code that expects `orders` to be iterable. ^[101]

What is happening

- In your `models.py`, `Order` belongs to `CustomerProfile` via `customer_id`, and you defined a backref `orders` on `CustomerProfile`:

```
customer = db.relationship("CustomerProfile", backref=db.backref("orders", lazy="dynamic"))
```

^[101]

- `customer_profile` (the object you likely created with `current_user.customer_profile`) is a **single profile object**, not its `.orders`.
- In `account.html`, you probably loop with something like:

```
{% for order in orders %}  
    ...  
{% endfor %}
```

which fails or shows nothing if `orders` is a single object instead of a query/list. ^{[102] [103]}

How to fix it

In your route, load the orders from the profile and pass those:

```
@app.route("/account")  
@login_required  
def account():  
    profile = current_user.customer_profile  
  
    if profile:  
        user_orders = profile.orders.all() # get list of Order objects  
    else:
```

```
user_orders = []

return render_template("account.html", user=current_user, orders=user_orders)
```

Now `orders` in the template is a list, and your `{% for order in orders %}` loop will render correctly.

✱

nvm its working now

Gut, dann ist alles im Lot. Wenn du als Nächstes z.B. konkrete Felder oder ein besseres Layout für die Account-Seite oder die Bestellliste brauchst, einfach den aktuellen Template-Code schicken.

generate a 20 products, add them do the database. use this data model. class `Product(db.Model)`:

```
__tablename__ = "product"

product_id = db.Column(db.Integer, primary_key=True)
producer_id = db.Column(db.Integer, db.ForeignKey("producer_profile.producer_id"), nullable=False)
category_id = db.Column(db.Integer, db.ForeignKey("category.category_id"), nullable=False)
name = db.Column(db.String(255), nullable=False)
description = db.Column(db.Text)
price = db.Column(db.Numeric(10, 2), nullable=False)
is_active = db.Column(db.Boolean, default=True, nullable=False)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

producer = db.relationship("ProducerProfile", back_populates="products")
category = db.relationship("Category", back_populates="products")

cart_items = db.relationship("CartItem", back_populates="product", lazy="dynamic")
order_items = db.relationship("OrderItem", back_populates="product", lazy="dynamic")
```

let me know which order table you need access to

To insert 20 products, the code needs valid `producer_id` and `category_id` values that already exist in your `producer_profile` and `category` tables. ^[117]

Assuming you already have at least one `ProducerProfile` and one `Category`, an example seeding script inside a Python shell with app context could look like:

```

from app import app, db
from models import Product, ProducerProfile, Category
from datetime import datetime
from decimal import Decimal

with app.app_context():
    producer = ProducerProfile.query.first()
    category = Category.query.first()

    products = [
        Product(
            producer_id=producer.producer_id,
            category_id=category.category_id,
            name="Organic Apples",
            description="Fresh organic apples from local orchards.",
            price=Decimal("3.49"),
        ),
        Product(
            producer_id=producer.producer_id,
            category_id=category.category_id,
            name="Free-range Eggs",
            description="Dozen free-range eggs from happy hens.",
            price=Decimal("4.99"),
        ),
        Product(
            producer_id=producer.producer_id,
            category_id=category.category_id,
            name="Sourdough Bread",
            description="Traditional sourdough bread baked daily.",
            price=Decimal("2.99"),
        ),
        Product(
            producer_id=producer.producer_id,
            category_id=category.category_id,
            name="Raw Honey",
            description="Unprocessed honey from local bees.",
            price=Decimal("5.50"),
        ),
        Product(
            producer_id=producer.producer_id,
            category_id=category.category_id,
            name="Goat Cheese",
            description="Soft goat cheese with mild flavor.",
            price=Decimal("6.20"),
        ),
        Product(
            producer_id=producer.producer_id,
            category_id=category.category_id,
            name="Cherry Tomatoes",
            description="Sweet cherry tomatoes, perfect for salads.",
            price=Decimal("2.30"),
        ),
        Product(
            producer_id=producer.producer_id,
            category_id=category.category_id,
            name="Baby Spinach",

```

```

        description="Tender baby spinach leaves.",
        price=Decimal("1.80"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Carrots",
        description="Crunchy carrots, ideal for snacking.",
        price=Decimal("1.50"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Potatoes",
        description="Waxy potatoes for roasting and salads.",
        price=Decimal("2.10"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Red Onions",
        description="Aromatic red onions for cooking and salads.",
        price=Decimal("1.60"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Zucchini",
        description="Fresh zucchini, great for grilling.",
        price=Decimal("1.90"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Cucumbers",
        description="Crisp cucumbers for salads and snacks.",
        price=Decimal("1.70"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Strawberries",
        description="Sweet, ripe strawberries.",
        price=Decimal("3.80"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Blueberries",
        description="Fresh blueberries full of antioxidants.",
        price=Decimal("4.20"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Whole Milk",

```

```

        description="Fresh whole milk from local dairy.",
        price=Decimal("1.40"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Yogurt",
        description="Natural yogurt with live cultures.",
        price=Decimal("1.10"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Herb Mix",
        description="Mixed fresh herbs (parsley, chives, dill).",
        price=Decimal("1.60"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Lettuce",
        description="Crisp head of lettuce.",
        price=Decimal("1.30"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Bell Peppers",
        description="Mixed red, yellow, and green bell peppers.",
        price=Decimal("2.70"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=category.category_id,
        name="Pasta Sauce",
        description="Homemade tomato basil pasta sauce.",
        price=Decimal("3.10"),
    ),
]

db.session.add_all(products)
db.session.commit()

```

If you want products spread across multiple producers and categories, say which `Category` and `ProducerProfile` rows you already have (e.g. IDs, or how you want them distributed), and the code can be adapted to that. Using `session.add_all` with a list of ORM objects is the recommended way to add many rows via Flask-SQLAlchemy. [\[118\]](#) [\[117\]](#)

✱

i dont have any producers or categories. create a producers (select an appropriate pplaceholder name), create a few logical categories (produce, milk products, eggs, fruit, meats)

To add one producer, 5 categories, and 20 products, run the following once in a Python shell with app context (same venv and project root as your app).^[132]

```
from app import app, db
from models import ProducerProfile, Category, Product, User, Address
from datetime import datetime
from decimal import Decimal

with app.app_context():
    # 1) Create a dummy user and address for the producer
    producer_user = User(
        email="producer@example.com",
        password_hash="dummy", # or a real hash if you want
        first_name="Local",
        last_name="Farmer",
        role="PRODUCER",
    )
    db.session.add(producer_user)
    db.session.flush() # get producer_user.id

    producer_address = Address(
        street="Hauptstraße 1",
        city="Berlin",
        zip="10115",
        country="Deutschland",
    )
    db.session.add(producer_address)
    db.session.flush() # get address_id

    producer = ProducerProfile(
        user_id=producer_user.id,
        address_id=producer_address.address_id,
        display_name="Biohof Sonnentäl",
        legal_name="Biohof Sonnentäl GmbH",
        tax_id="DE123456789",
        contact_email="kontakt@biohof-sonnentäl.de",
        contact_phone="+49 30 1234567",
        verification_status="verified",
    )
    db.session.add(producer)

    # 2) Create categories
    category_names = [
        "Produce",          # Gemüse etc.
        "Milk products",    # Milch, Käse, Joghurt
        "Eggs",
        "Fruit",
        "Meats",
```

```

]
categories = []
for name in category_names:
    cat = Category(name=name)
    db.session.add(cat)
    categories.append(cat)

db.session.flush() # get category_ids

# Helper: pick categories by name
cat_by_name = {c.name: c for c in categories}

# 3) Create 20 products
products = [
    # Produce
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Produce"].category_id,
        name="Carrots",
        description="Crunchy organic carrots from the farm.",
        price=Decimal("1.80"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Produce"].category_id,
        name="Zucchini",
        description="Fresh zucchini, ideal for grilling.",
        price=Decimal("2.10"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Produce"].category_id,
        name="Cherry Tomatoes",
        description="Sweet cherry tomatoes for salads and snacks.",
        price=Decimal("2.90"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Produce"].category_id,
        name="Baby Spinach",
        description="Tender baby spinach leaves.",
        price=Decimal("2.40"),
    ),

    # Milk products
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Milk products"].category_id,
        name="Whole Milk 1L",
        description="Fresh whole milk from grass-fed cows.",
        price=Decimal("1.40"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Milk products"].category_id,
        name="Natural Yogurt",

```

```

        description="Plain yogurt with live cultures.",
        price=Decimal("1.20"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Milk products"].category_id,
        name="Goat Cheese",
        description="Soft goat cheese with mild flavor.",
        price=Decimal("5.90"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Milk products"].category_id,
        name="Butter",
        description="Creamy farm butter, ideal for baking.",
        price=Decimal("2.30"),
    ),

    # Eggs
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Eggs"].category_id,
        name="Free-range Eggs (10 pcs)",
        description="Eggs from free-range hens.",
        price=Decimal("3.50"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Eggs"].category_id,
        name="Organic Eggs (6 pcs)",
        description="Certified organic eggs.",
        price=Decimal("3.10"),
    ),

    # Fruit
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Fruit"].category_id,
        name="Apples",
        description="Mixed varieties of fresh apples.",
        price=Decimal("3.20"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Fruit"].category_id,
        name="Pears",
        description="Sweet, ripe pears.",
        price=Decimal("3.40"),
    ),
    Product(
        producer_id=producer.producer_id,
        category_id=cat_by_name["Fruit"].category_id,
        name="Strawberries",
        description="Fresh strawberries (250g punnet).",
        price=Decimal("3.90"),
    ),

```

```

Product(
    producer_id=producer.producer_id,
    category_id=cat_by_name["Fruit"].category_id,
    name="Blueberries",
    description="Blueberries rich in antioxidants.",
    price=Decimal("4.20"),
),

# Meats
Product(
    producer_id=producer.producer_id,
    category_id=cat_by_name["Meats"].category_id,
    name="Beef Mince",
    description="Minced beef from pasture-raised cattle.",
    price=Decimal("7.50"),
),
Product(
    producer_id=producer.producer_id,
    category_id=cat_by_name["Meats"].category_id,
    name="Pork Sausages",
    description="Farm-style pork sausages.",
    price=Decimal("5.20"),
),
Product(
    producer_id=producer.producer_id,
    category_id=cat_by_name["Meats"].category_id,
    name="Chicken Breast",
    description="Skinless chicken breast fillets.",
    price=Decimal("6.80"),
),
Product(
    producer_id=producer.producer_id,
    category_id=cat_by_name["Meats"].category_id,
    name="Smoked Ham",
    description="Traditional smoked ham.",
    price=Decimal("4.90"),
),
Product(
    producer_id=producer.producer_id,
    category_id=cat_by_name["Meats"].category_id,
    name="Bone Broth",
    description="Slow-cooked beef bone broth.",
    price=Decimal("3.70"),
),
]

db.session.add_all(products)
db.session.commit()

```

This uses `session.add_all` to insert multiple objects in one go, which is idiomatic for Flask-SQLAlchemy seeding scripts. [\[133\]](#) [\[132\]](#)

✱✱

does this script need to run once and the code creates these instances, saves them in the database? i could technically delete the code in this file afterwards

Exactly: you only need to run that code **once** inside an app context; it creates the rows and saves them in the database, then you can delete or comment out the script. ^[134]

How to use it once

- Put the code either:
 - Temporarily into a separate file like `seed_data.py`, or
 - Run it manually in a Python shell with `with app.app_context(): ....`

Example with a separate file `seed_data.py`:

```
from app import app, db
from models import ProducerProfile, Category, Product, User, Address
from decimal import Decimal

with app.app_context():
    # ... seeding code here ...
    db.session.commit()
```

Run once:

```
python seed_data.py
```

After it succeeds, the data is stored in `farmersmarket.db`, and your app can query it normally. You can then remove `seed_data.py` or keep it for future resets; the inserted data does **not** disappear when you delete the script file. ^[135] ^[134]

✱

i wrote this code in `init_db.py`

Yes, that works the same way: `init_db.py` is just your seeding script file.

Minimal pattern for `init_db.py`:

```
from app import app, db
from models import ProducerProfile, Category, Product, User, Address
from decimal import Decimal

with app.app_context():
    # create users / producer / categories / products here
```

```
# ...  
db.session.commit()
```

Then run it once from your project root (with venv active):

```
python init_db.py
```

After it runs without errors, the producer, categories, and products are in your SQLite database; you do not need to keep or import `init_db.py` in your main app for the data to stay. [\[136\]](#) [\[137\]](#)

✱

ok i rant `init_db.py` once, then `app.py`. in the products page it doesnt show anything yet

Right now your `/products` route just renders a static template without querying the database, so the products you inserted are never passed to the page. [\[138\]](#)

1. Update the `/products` route

Change your route from:

```
@app.route("/products")  
def products():  
    return render_template("products.html")
```

to something like:

```
from models import Product, Category  
  
@app.route("/products")  
def products():  
    products = Product.query.filter_by(is_active=True).all()  
    return render_template("products.html", products=products)
```

Optionally also load categories if you want filters:

```
@app.route("/products")  
def products():  
    products = Product.query.filter_by(is_active=True).all()  
    categories = Category.query.all()  
    return render_template("products.html", products=products, categories=categories)
```

2. Render products in products.html

In `templates/products.html`, loop over `products`:

```
{% extends "base.html" %}
{% block title %}Produkte{% endblock %}

{% block content %}
<h1>Produkte</h1>

{% if products %}
<div class="product-list">
  {% for product in products %}
    <div class="product-card">
      <h2>{{ product.name }}</h2>
      <p>{{ product.description }}</p>
      <p><strong>{{ product.price }} €</strong></p>
      <p>Kategorie: {{ product.category.name }}</p>
      <p>Produzent: {{ product.producer.display_name }}</p>
    </div>
  {% endfor %}
</div>
{% else %}
<p>Derzeit sind keine Produkte verfügbar.</p>
{% endif %}
{% endblock %}
```

Now the 20 seeded products will be pulled from the DB and displayed on the products page.

✱

is it better to render products into html or the route

The route should **load the products**, and the HTML template should **render** them; both parts are needed but with different responsibilities. [\[139\]](#) [\[140\]](#)

What belongs in the route

- Query the database and prepare data:

```
@app.route("/products")
def products():
    products = Product.query.filter_by(is_active=True).all()
    return render_template("products.html", products=products)
```

- Keep this layer for logic: filtering, sorting, pagination, etc. [\[140\]](#) [\[141\]](#)

What belongs in the template

- Display the data passed from the route:

```
{% for product in products %}
  <h2>{{ product.name }}</h2>
  <p>{{ product.description }}</p>
  <p>{{ product.price }} €</p>
{% endfor %}
```

- Keep this layer for presentation (HTML/CSS), not for DB queries.^[139]

✱

now create an interface for businesses, the most right button in the navbar is for business. when you click on it, you are again first asked an email, then password for login or the signup fields if user is not found. also ask business relevant information in order to authenticate real businesses. this page should have following functionality: add category, add products, edit business information like opening hours, adress, or other info i cant think of right now.

suggest core functionalities for this page

For businesses you essentially need a **producer cockpit**: login/signup plus a simple dashboard to manage profile, categories, and products.^[142] ^[143]

Auth flow for businesses

- Separate business login route, e.g. `/business/login`, linked from the rightmost navbar button "Für Betriebe" (or similar label).
- Same pattern as customers:
 - Step 1: ask for email.
 - Step 2: if user with `role="PRODUCER"` exists → ask for password; if not → show signup form.^[144]
- On signup, collect business-specific data along with the `User` record and create a `ProducerProfile` + `Address` row.

Business signup / profile fields

For authenticating real businesses and making the shop useful:

- Company identity:
 - Display name (public), legal name, tax/VAT ID, contact email, phone. ^[142]
- Location & opening hours:
 - Street, ZIP, city, country.
 - Opening hours per weekday, plus “by appointment” or “closed” flags.
- Business metadata:
 - Short description (“Biohof in Brandenburg mit Fokus auf Gemüse”),
 - Delivery / pickup options (only pickup, local delivery radius, etc.).

These map well onto your existing `ProducerProfile` + `Address` model.

Dashboard core sections

After login, show a **Business dashboard** page (e.g. `/business/dashboard`) with clear sections:

- Profile overview:
 - Show current display name, address, opening times, contact data with an “Edit profile” button.
- Categories management:
 - List categories this producer uses and allow:
 - Add new category (local to this producer or pick from global category list).
 - Optionally deactivate a category for this producer.
- Product management:
 - List of products with: name, category, price, active/inactive.
 - Actions: add product, edit product, deactivate (soft delete) product. ^[143] ^[142]

Business actions (forms/routes)

Key functionalities to implement:

- Edit business information:
 - Route: `/business/profile` with a `ProducerProfileForm` (display_name, description, opening_hours, contact_email, phone, address fields).
- Add/edit categories:
 - Route `/business/categories`
 - Simple form: category name, maybe type (Produce, Fruit, etc.).
- Add/edit products:

- Route `/business/products` (list) and `/business/products/new`, `/business/products/<id>/edit`.
- Form fields: name, category (select), description, price, is_active.

Useful UX additions

- Clear status indicator: "Your shop is live / pending verification / incomplete profile".
- Simple stats snapshot: number of active products, last order date, total orders (later).
- Help text: short explanation at top:
 - "Hier verwaltest du dein Profil und deine Produkte. Änderungen sind sofort im Marktplatz sichtbar."^[145] ^[142]

If you want, next step can be: sketch the routes (`/business/login`, `/business/dashboard`, `/business/profile`, `/business/products`) and a minimal `business_dashboard.html` layout reflecting these sections.

✱

generate a route that does this email first workflow

```
@app.route("/business")
def business():
    return render_template("business.html")
```

this is the current route

You can reuse dein bestehendes E-Mail-/Passwort-Stufen-Muster und auf `role="PRODUCER"` filtern.

```
from flask import request, redirect, render_template, url_for
from flask_login import login_user
from models import User
from forms import EmailOnlyLoginForm, PasswordOnlyLoginForm, SignupForm
from werkzeug.security import generate_password_hash, check_password_hash

@app.route("/business", methods=["GET", "POST"])
def business():
    step = request.args.get("step", "email") # "email", "password", "signup"
    email = request.args.get("email", "").strip().lower()

    # 1) Email step
    if step == "email":
        form = EmailOnlyLoginForm()
        if form.validate_on_submit():
            email = form.email.data.strip().lower()
            user = User.query.filter_by(email=email, role="PRODUCER").first()
            if user:
                # Producer exists -> ask for password
                return redirect(url_for("business", step="password", email=email))
            else:
```

```

        # No producer with this email -> signup
        return redirect(url_for("business", step="signup", email=email))
    return render_template("business.html", step="email", email=email, form=form)

# 2) Password step (existing producer)
if step == "password":
    form = PasswordOnlyLoginForm()
    if form.validate_on_submit():
        password = form.password.data
        user = User.query.filter_by(email=email, role="PRODUCER").first()
        if user and check_password_hash(user.password_hash, password):
            login_user(user)
            return redirect(url_for("business_dashboard")) # späteres Dashboard
        else:
            form.password.errors.append("Falsches Passwort oder kein Produzentenkonto")
    return render_template("business.html", step="password", email=email, form=form)

# 3) Signup step (new producer)
if step == "signup":
    form = SignupForm()
    if form.validate_on_submit():
        first_name = form.first_name.data
        last_name = form.last_name.data
        username = form.username.data # falls du es später nutzt
        password = form.password.data

        user = User(
            email=email,
            password_hash=generate_password_hash(password),
            first_name=first_name,
            last_name=last_name,
            role="PRODUCER",
        )
        db.session.add(user)
        db.session.commit()

        # TODO: hier direkt ProducerProfile + Address erstellen, wenn du willst

        login_user(user)
        return redirect(url_for("business_dashboard"))
    return render_template("business.html", step="signup", email=email, form=form)

# Fallback: behandle alles andere wie signup
form = SignupForm()
return render_template("business.html", step="signup", email=email, form=form)

```

In `business.html` kannst du analog zur Kunden-Loginseite je nach `step` das passende Formular anzeigen.

create a new form SignupFormProducers, right now you used the signupform for customers, they only need email name and adress, however producers need to input more data to authenticate themselves. create a new form for forms.py and name it SignupFormProducers

Für Produzenten brauchst du ein erweitertes Formular, das auf deinem bestehenden SignupForm aufbaut, aber mehr Felder für Unternehmensdaten enthält. ^[146]

Füge in forms.py z.B. folgendes hinzu:

```
class SignupFormProducers(FlaskForm):
    # Persönliche Daten
    first_name = StringField(
        "Vorname", validators=[InputRequired(), Length(min=2, max=50)]
    )
    last_name = StringField(
        "Nachname", validators=[InputRequired(), Length(min=2, max=50)]
    )

    # Account-Daten
    email = StringField(
        "E-Mail",
        validators=[InputRequired("Bitte E-Mail eingeben."), Email("Bitte gültige E-Mail
    )
    password = PasswordField(
        "Passwort",
        validators=[InputRequired("Bitte Passwort eingeben."),
                    Length(min=6, message="Passwort muss mindestens 6 Zeichen lang sein."
    )
    password_confirm = PasswordField(
        "Passwort bestätigen",
        validators=[InputRequired("Bitte Passwort bestätigen."),
                    EqualTo("password", message="Passwörter stimmen nicht überein.")],
    )

    # Unternehmensdaten
    display_name = StringField(
        "Anzeigename (öffentlich)",
        validators=[InputRequired(), Length(min=2, max=255)],
    )
    legal_name = StringField(
        "Rechtlicher Name (optional)",
        validators=[Length(max=255)],
    )
    tax_id = StringField(
        "Steuer-/USt-ID",
        validators=[InputRequired(), Length(min=5, max=50)],
    )
    contact_email = StringField(
```

```

        "Kontakt-E-Mail",
        validators=[InputRequired(), Email()],
    )
    contact_phone = StringField(
        "Telefon (optional)",
        validators=[Length(max=50)],
    )

    # Adresse des Betriebs
    street = StringField(
        "Straße und Hausnummer",
        validators=[InputRequired(), Length(min=3, max=255)],
    )
    zip_code = StringField(
        "PLZ",
        validators=[
            InputRequired(),
            Length(min=4, max=10),
            Regexp(r"^[0-9]+$", message="Bitte nur Ziffern eingeben."),
        ],
    )
    city = StringField(
        "Stadt",
        validators=[InputRequired(), Length(min=2, max=100)],
    )
    country = StringField(
        "Land",
        validators=[InputRequired(), Length(min=2, max=100)],
    )

    # Öffnungszeiten (einfach gehalten)
    opening_hours = StringField(
        "Öffnungszeiten (frei formuliert)",
        validators=[Length(max=255)],
    )

    submit = SubmitField("Als Betrieb registrieren")

```

Damit kannst du in der Business-Signup-Route statt `SignupForm` dann `SignupFormProducers` verwenden und daraus `User (role="PRODUCER")`, `Address` und `ProducerProfile` befüllen. [\[147\]](#) [\[146\]](#)

✱

dont you think the form needs to match the variables in the datamodel?

Yes, ideally the form fields should map cleanly to your data model so you can save them without awkward conversions. [\[148\]](#)

Match to your current models

From your `models.py`:

- User: email, password_hash, first_name, last_name, role.
- Address: street, city, zip, country.
- ProducerProfile:
 - user_id, address_id
 - display_name, legal_name, tax_id, contact_email, contact_phone, verification_status.

So the producer signup form should give you **exactly** these attributes (plus password confirmation):

- For User: first_name, last_name, email, password, password_confirm.
- For Address: street, zip, city, country.
- For ProducerProfile: display_name, legal_name (optional), tax_id, contact_email, contact_phone.

The `SignupFormProducers` from the last message already matches those field names conceptually (you only need to remember that `zip_code` from the form maps to `zip` in the model when you create `Address`).

✱

class ProducerProfile(db.Model):

```
__tablename__ = "producer_profile"

producer_id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True)
address_id = db.Column(db.Integer, db.ForeignKey("address.address_id"), nullable=True)
display_name = db.Column(db.String(255), nullable=False)
legal_name = db.Column(db.String(255), nullable=True)
tax_id = db.Column(db.String(50), nullable=True)
contact_email = db.Column(db.String(255), nullable=False)
contact_phone = db.Column(db.String(50), nullable=True)
verification_status = db.Column(db.String(20), nullable=False, default="pending")
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

user = db.relationship("User", back_populates="producer_profile")
products = db.relationship("Product", back_populates="producer", lazy="dynamic")
address = db.relationship("Address", back_populates="producers")
```

this is the datamodel for producers

Yes, and the producer signup form should give you values for all non-nullable fields in this model (plus what you need for `User` and `Address`). [\[149\]](#)

Fields your form must cover

From `ProducerProfile`:

- Required (`nullable=False`):
 - `display_name`
 - `contact_email`
 - `verification_status` has a default "pending", so you do not need a form field.
- Optional:
 - `legal_name`
 - `tax_id`
 - `contact_phone`
 - `address_id` can be null, but in your business flow you probably want an address from the start.

From `User` (producer account):

- `email`, `password_hash`, `first_name`, `last_name`, `role="PRODUCER"`.^[149]

From `Address`:

- `street`, `zip`, `city`, `country`.^[149]

So `SignupFormProducers` is correct if it has:

- `first_name`, `last_name`, `email`, `password`, `password_confirm`.
- `display_name`, `legal_name`, `tax_id`, `contact_email`, `contact_phone`.
- `street`, `zip_code` (mapped to `zip`), `city`, `country`.

Then in your `/business` signup step you:

1. Create `User` with `role="PRODUCER"`.
2. Create `Address` from the address fields.
3. Create `ProducerProfile` using the above `ProducerProfile` fields and linking `user_id` and `address_id`.

✱

now according to that, create a form

Yes, the producer form should mirror what you need to populate `User`, `Address`, and `ProducerProfile` without extra fields.^[150]

SignupFormProducers for forms.py

```
from flask_wtf import FlaskForm
from wtforms import StringField, PasswordField, SubmitField
from wtforms.validators import (
    InputRequired,
    Length,
    Email,
    EqualTo,
    Regexp,
)

class SignupFormProducers(FlaskForm):
    # User data
    first_name = StringField(
        "Vorname",
        validators=[InputRequired(), Length(min=2, max=50)],
    )
    last_name = StringField(
        "Nachname",
        validators=[InputRequired(), Length(min=2, max=50)],
    )
    email = StringField(
        "Login-E-Mail",
        validators=[InputRequired(), Email(), Length(max=255)],
    )
    password = PasswordField(
        "Passwort",
        validators=[
            InputRequired(),
            Length(min=6, message="Passwort muss mindestens 6 Zeichen haben."),
        ],
    )
    password_confirm = PasswordField(
        "Passwort bestätigen",
        validators=[
            InputRequired(),
            EqualTo("password", message="Passwörter stimmen nicht überein."),
        ],
    )

    # ProducerProfile data
    display_name = StringField(
        "Anzeigename (öffentlich)",
        validators=[InputRequired(), Length(min=2, max=255)],
    )
    legal_name = StringField(
        "Rechtlicher Name (optional)",
        validators=[Length(max=255)],
    )
    tax_id = StringField(
        "Steuer-/USt-ID (optional)",
        validators=[Length(max=50)],
    )
    contact_email = StringField(
        "Kontakt-E-Mail (für Kunden sichtbar)",
```



```

        validators=[InputRequired(), Email(), Length(max=255)],
    )
    contact_phone = StringField(
        "Telefon (optional)",
        validators=[Length(max=50)],
    )

    # Address data (for Address model)
    street = StringField(
        "Straße und Hausnummer",
        validators=[InputRequired(), Length(min=3, max=255)],
    )
    zip_code = StringField(
        "PLZ",
        validators=[
            InputRequired(),
            Length(min=4, max=10),
            Regexp(r"^[0-9]+$", message="Bitte nur Ziffern eingeben."),
        ],
    )
    city = StringField(
        "Stadt",
        validators=[InputRequired(), Length(min=2, max=100)],
    )
    country = StringField(
        "Land",
        validators=[InputRequired(), Length(min=2, max=100)],
    )

    submit = SubmitField("Als Produzent registrieren")

```

How this maps to models

- User: first_name, last_name, email, password → hash and set role="PRODUCER".
- Address: street, zip ← zip_code, city, country.
- ProducerProfile: display_name, legal_name, tax_id, contact_email, contact_phone, plus user_id and address_id from the created records. [\[151\]](#) [\[152\]](#)

✱✱

where should i ask for the adress, when i do it in signupformproducers, can the fields upload the data in the adress table or is everyhting uploaded in the producers table?

The address fields can live in SignupFormProducers, but they will not "automatically" go to the address table; you decide in the view which model each field populates. [\[165\]](#) [\[166\]](#)

Where to ask for address

Put the address inputs directly into `SignupFormProducers`, because they are part of the producer onboarding flow and you need them only there.

In the template you render them like normal form fields alongside the producer data. ^[167]

How the data is saved

In your route (example `/signup/producer`):

```
@app.route("/signup/producer", methods=["GET", "POST"])
def signup_producer():
    form = SignupFormProducers()
    if form.validate_on_submit():
        # 1) User
        user = User(
            first_name=form.first_name.data,
            last_name=form.last_name.data,
            email=form.email.data,
            role="PRODUCER",
        )
        user.set_password(form.password.data)
        db.session.add(user)

        # 2) Address
        address = Address(
            street=form.street.data,
            zip=form.zip_code.data,
            city=form.city.data,
            country=form.country.data,
        )
        db.session.add(address)

        # Flush so user.id and address.address_id are available
        db.session.flush()

        # 3) ProducerProfile
        producer = ProducerProfile(
            user_id=user.id,
            address_id=address.address_id,
            display_name=form.display_name.data,
            legal_name=form.legal_name.data,
            tax_id=form.tax_id.data,
            contact_email=form.contact_email.data,
            contact_phone=form.contact_phone.data,
            # verification_status uses default="pending"
        )
        db.session.add(producer)

        db.session.commit()
        # login_user(user) or redirect, etc.
        return redirect(url_for("index"))

    return render_template("signup_producer.html", form=form)
```

- The **same form** feeds three different models: User, Address, and ProducerProfile.
- Which table gets what is controlled entirely by how you construct the model objects in the view before committing.^{[168] [169]}

**

@app.route("/business", methods=["GET", "POST"])

```
def business():
```

```
    step = request.args.get("step", "email")
```

```
    email = request.args.get("email", "").strip().lower()
```

```
    if email:
        form = EmailOnlyLoginForm()
        if form.validate_on_submit():
            email = form.email.data.strip().lower()
            user = User.query.filter_by(email=email, role="PRODUCER").first()
            if user:
                return redirect(url_for("business", step="password", email=email))
            else:
                return redirect(url_for("business", step="signup", email=email))
        return render_template("business.html", step="email", email=email, form=form)

    if step == "password":
        form = PasswordOnlyLoginForm()
        if form.validate_on_submit():
            password = form.password.data
            user = User.query.filter_by(email=email, role="PRODUCER").first()
            if user and check_password_hash(user.password_hash, password):
                login_user(user)
                return redirect(url_for("home")) #wenn logged in, switch to business dashboard
            else:
                form.password.errors.append("Falsches Passwort.")
        return render_template("business.html", step="password", email=email, form=form)

    if step == "signup":
        form = SignupFormProducers()
        if form.validate_on_submit():
            username = form.username.data
            password = form.password.data

            user = User(
                first_name = form.first_name.data
                last_name = form.last_name.data
                email=form.email.data,
                role="PRODUCER"
            )
            user.set_password(form.password.data)
            db.session.add(user)
```

```

address = Address(
    street=form.street.data,
    city=form.city.data,
    zip=form.zip.data,
    country=form.country.data
)
db.session.commit()
db.session.flush()

producer_profile = ProducerProfile(
    user_id=user.id,
    address_id=address.address_id,
    display_name=form.display_name.data,
    legal_name=form.legal_name.data,
    tax_id=form.tax_id.data,
    contact_email=form.contact_email.data,
    contact_phone=form.contact_phone.data
)
db.session.add(producer_profile)

db.session.commit()
login_user(user)
return redirect(url_for("business_dashboard"))
return render_template("business.html", step="signup", email=email, form=form)
return render_template("business.html")

```

now check this workflow please

There are a few small bugs, but the overall flow (email → password or signup with User + Address + ProducerProfile) is fine. [\[181\]](#) [\[182\]](#)

Issues in your current code

- In the signup block you read `username = form.username.data`, but `SignupFormProducers` has no `username` field; this line can be removed.
- When creating `User` you miss a comma between `first_name` and `last_name`, which will cause a syntax error.
- When creating `Address` you access `form.zip.data`, but the form field is named `zip_code`; this will raise an `AttributeError`.
- You call `db.session.commit()` *before* `db.session.flush()`, and you never `add(address)` to the session, so `address.address_id` may be `None` or `address` may not be persisted. [\[183\]](#) [\[184\]](#)

Corrected / improved workflow

```

@app.route("/business", methods=["GET", "POST"])
def business():
    step = request.args.get("step", "email")
    email = request.args.get("email", "").strip().lower()

```

```

# STEP 1: ask for email
if step == "email":
    form = EmailOnlyLoginForm()
    if form.validate_on_submit():
        email = form.email.data.strip().lower()
        user = User.query.filter_by(email=email, role="PRODUCER").first()
        if user:
            return redirect(url_for("business", step="password", email=email))
        else:
            return redirect(url_for("business", step="signup", email=email))
    return render_template("business.html", step="email", email=email, form=form)

# STEP 2: password for existing producer
if step == "password":
    form = PasswordOnlyLoginForm()
    if form.validate_on_submit():
        password = form.password.data
        user = User.query.filter_by(email=email, role="PRODUCER").first()
        if user and check_password_hash(user.password_hash, password):
            login_user(user)
            return redirect(url_for("home")) # später business_dashboard
        else:
            form.password.errors.append("Falsches Passwort.")
    return render_template("business.html", step="password", email=email, form=form)

# STEP 3: signup for new producer
if step == "signup":
    form = SignupFormProducers()
    if form.validate_on_submit():

        # 1) User
        user = User(
            first_name=form.first_name.data,
            last_name=form.last_name.data,
            email=form.email.data.strip().lower(),
            role="PRODUCER",
        )
        user.set_password(form.password.data)
        db.session.add(user)

        # 2) Address
        address = Address(
            street=form.street.data,
            city=form.city.data,
            zip=form.zip_code.data,
            country=form.country.data,
        )
        db.session.add(address)

        # flush to get ids without committing yet
        db.session.flush()

        # 3) ProducerProfile
        producer_profile = ProducerProfile(
            user_id=user.id,
            address_id=address.address_id,

```

```

        display_name=form.display_name.data,
        legal_name=form.legal_name.data or None,
        tax_id=form.tax_id.data or None,
        contact_email=form.contact_email.data.strip().lower(),
        contact_phone=form.contact_phone.data or None,
    )
    db.session.add(producer_profile)

    # final commit
    db.session.commit()

    login_user(user)
    return redirect(url_for("business_dashboard"))

    return render_template("business.html", step="signup", email=email, form=form)

# fallback
return redirect(url_for("business", step="email"))

```

Key points:

- Use `step` consistently, including when `email` is not in the query (default to `email` step).
- Add `user` and `address` to the same session, flush once, then create `ProducerProfile`, and commit at the end of the transaction. [\[185\]](#) [\[183\]](#)

✱

{% extends "base.html" %}

```
{% block title %}Account{% endblock %}
```

```
{% block content %}
```

Mein Konto

Eingeloggt als: {{ session["user_email"] }}

Persönliche Daten

Name: {{ user.first_name }} {{ user.last_name }}

...

E-Mail: {{ user.email }}

...

Konto angelegt am: {{ user.created_at.strftime("%d.%m.%Y") }}

...

Standard-Lieferadresse

...

{{ user.address }}

{{ user.zip_code }} {{ user.city }}

Adresse bearbeiten

Bestellungen

{% if orders %}

 {% for order in orders %}

- `` **Bestellung #{{ order.id }}** – `` {{ order.created_at.strftime("%d.%m.%Y") }} – Gesamt: {{ order.total }} € `` Details ``

 {% endfor %}

{% else %}

Du hast noch keine Bestellungen.

{% endif %} Logout

{% endblock %}

this is the user account dashboard, create a similar dashboard for business, i dont have an html code for that yet

For Produzenten brauchst du ein eigenes Dashboard, das die Produzentendaten, Betriebsadresse und Produkte/Bestellungen zeigt, ähnlich strukturiert wie dein Kundenkonto.

[196]

Beispiel: business_dashboard.html

```
{% extends "base.html" %}
{% block title %}Business-Dashboard{% endblock %}

{% block content %}
    <h1>Mein Betrieb</h1>

    <p>Eingeloggt als: {{ current_user.email }}</p>

    {{# PRODZENTEN-PROFIL #}}
    <section class="account-section">
        <h2>Profil</h2>
        <p><strong>Öffentlicher Name:</strong> {{ producer.display_name }}</p>
        <p><strong>Rechtlicher Name:</strong>
            {{ producer.legal_name or "nicht hinterlegt" }}
        </p>
        <p><strong>Kontakt-E-Mail:</strong> {{ producer.contact_email }}</p>
        <p><strong>Telefon:</strong> {{ producer.contact_phone or "nicht hinterlegt" }}</p>
        <p><strong>Status:</strong> {{ producer.verification_status }}</p>
        <p><strong>Profil angelegt am:</strong>
            {{ producer.created_at.strftime("%d.%m.%Y") }}
        </p>
    </section>
{{/PRODZENTEN-PROFIL}}
```

```

    <a href="{{ url_for('edit_producer_profile') }}">Profil bearbeiten</a>
</section>

{{# BETRIEBS-ADRESSE #}}
<section class="account-section">
    <h2>Betriebsadresse</h2>
    {% if producer.address %}
        <p>{{ producer.address.street }}</p>
        <p>{{ producer.address.zip }} {{ producer.address.city }}</p>
        <p>{{ producer.address.country }}</p>
    {% else %}
        <p>Noch keine Adresse hinterlegt.</p>
    {% endif %}
    <a href="{{ url_for('edit_producer_address') }}">Adresse bearbeiten</a>
</section>

{{# PRODUKTE #}}
<section class="account-section">
    <h2>Produkte</h2>
    {% if products %}
        <ul>
            {% for product in products %}
                <li>
                    <strong>{{ product.name }}</strong> -
                    {{ product.price }} € -
                    Status: {{ product.status }}
                    <a href="{{ url_for('edit_product', product_id=product.id) }}">Bearbeiten</a>
                </li>
            {% endfor %}
        </ul>
    {% else %}
        <p>Du hast noch keine Produkte angelegt.</p>
    {% endif %}
    <a href="{{ url_for('create_product') }}">Neues Produkt anlegen</a>
</section>

{{# BESTELLUNGEN #}}
<section class="account-section">
    <h2>Bestellungen für deinen Betrieb</h2>
    {% if orders %}
        <ul>
            {% for order in orders %}
                <li>
                    <strong>Bestellung #{{ order.id }}</strong> -
                    {{ order.created_at.strftime("%d.%m.%Y") }} -
                    Gesamt: {{ order.total }} €
                    <a href="{{ url_for('business_order_detail', order_id=order.id) }}">Details</a>
                </li>
            {% endfor %}
        </ul>
    {% else %}
        <p>Es liegen noch keine Bestellungen vor.</p>
    {% endif %}
</section>

<form action="{{ url_for('logout') }}" method="post">

```



```
<button type="submit">Logout</button>
</form>
{% endblock %}
```

Erwartete Daten aus der Route

In deiner `business_dashboard`-Route solltest du typischerweise übergeben:

- `producer = current_user.producer_profile`
- `products = producer.products.all()`
- `orders = ...` (nur Bestellungen, die Produkte dieses Produzenten enthalten, je nach deinem Order-Modell gefiltert). [\[197\]](#) [\[198\]](#)



what do i need to rename or create now since `url_for()` are underlined

Those underlined `url_for()` calls mean there is no matching route (endpoint) with that name yet, or the parameters don't match. [\[211\]](#) [\[212\]](#)

Endpoints you are using in the template

From `business_dashboard.html` you currently reference:

- `url_for('edit_producer_profile')`
- `url_for('edit_producer_address')`
- `url_for('edit_product', product_id=product.id)`
- `url_for('create_product')`
- `url_for('business_order_detail', order_id=order.id)`
- `url_for('logout')` (this you likely already have)

All of these must correspond to real view function names in your Flask app.

What to create or rename

1. **Either create these routes**, for example:

```
@app.route("/business/dashboard")
@login_required
def business_dashboard():
    producer = current_user.producer_profile
    products = producer.products.all() if producer else []
    orders = [] # TODO: query only orders for this producer
    return render_template(
        "business_dashboard.html",
        producer=producer,
        products=products,
```

```

        orders=orders,
    )

@app.route("/business/profile/edit", methods=["GET", "POST"])
@login_required
def edit_producer_profile():
    ...

@app.route("/business/address/edit", methods=["GET", "POST"])
@login_required
def edit_producer_address():
    ...

@app.route("/business/products/new", methods=["GET", "POST"])
@login_required
def create_product():
    ...

@app.route("/business/products/<int:product_id>/edit", methods=["GET", "POST"])
@login_required
def edit_product(product_id):
    ...

@app.route("/business/orders/<int:order_id>")
@login_required
def business_order_detail(order_id):
    ...

```

2. Or rename the endpoints in the template to match existing route functions.

If you already have e.g. `@app.route("/product/<int:id>/edit") def product_edit(id):`, then change the template to `url_for('product_edit', id=product.id)` instead. [\[213\]](#) [\[214\]](#)

As a rule: the string in `url_for('...')` must equal the **Python function name** (or a custom endpoint= you set) for the corresponding `@app.route`.

✱✱

you mean rename this route:

```

@app.route("/products")
def products():
    products = Product.query.filter_by(is_active=True).all()
    categories = Category.query.all()
    return render_template("products.html", products=products, categories=categories)

```

however i dont want to be editing products with this route, since customers will use this route now businesses. then i guess i have to create new "business routes" which let the user then interact with the database (edit the database). create the necessary route for the html to work

Yes, keep `/products` for customers and add separate **business routes** for managing producer data. [\[226\]](#)

Business dashboard route

```
@app.route("/business/dashboard")
@login_required
def business_dashboard():
    producer = current_user.producer_profile
    if not producer:
        return redirect(url_for("business", step="signup", email=current_user.email))

    products = producer.products.all()
    # TODO: filter orders for this producer only
    orders = []

    return render_template(
        "business_dashboard.html",
        producer=producer,
        products=products,
        orders=orders,
    )
```

Edit producer profile

```
@app.route("/business/profile/edit", methods=["GET", "POST"])
@login_required
def edit_producer_profile():
    producer = current_user.producer_profile
    if not producer:
        return redirect(url_for("business_dashboard"))

    form = EditProducerProfileForm(obj=producer)
    if form.validate_on_submit():
        producer.display_name = form.display_name.data
        producer.legal_name = form.legal_name.data or None
        producer.contact_email = form.contact_email.data
        producer.contact_phone = form.contact_phone.data or None
        db.session.commit()
        return redirect(url_for("business_dashboard"))

    return render_template("edit_producer_profile.html", form=form)
```

Edit producer address

```
@app.route("/business/address/edit", methods=["GET", "POST"])
@login_required
def edit_producer_address():
    producer = current_user.producer_profile
    if not producer:
        return redirect(url_for("business_dashboard"))

    address = producer.address
    form = EditAddressForm(obj=address)
    if form.validate_on_submit():
```

```

        address.street = form.street.data
        address.zip = form.zip_code.data
        address.city = form.city.data
        address.country = form.country.data
        db.session.commit()
        return redirect(url_for("business_dashboard"))

    return render_template("edit_producer_address.html", form=form)

```

Create / edit products (business only)

```

@app.route("/business/products/new", methods=["GET", "POST"])
@login_required
def create_product():
    producer = current_user.producer_profile
    if not producer:
        return redirect(url_for("business_dashboard"))

    form = ProductForm()
    if form.validate_on_submit():
        product = Product(
            name=form.name.data,
            description=form.description.data,
            price=form.price.data,
            is_active=True,
            producer_id=producer.producer_id,
        )
        db.session.add(product)
        db.session.commit()
        return redirect(url_for("business_dashboard"))

    return render_template("product_form.html", form=form)

@app.route("/business/products/<int:product_id>/edit", methods=["GET", "POST"])
@login_required
def edit_product(product_id):
    producer = current_user.producer_profile
    product = Product.query.filter_by(
        id=product_id,
        producer_id=producer.producer_id
    ).first_or_404()

    form = ProductForm(obj=product)
    if form.validate_on_submit():
        product.name = form.name.data
        product.description = form.description.data
        product.price = form.price.data
        product.is_active = form.is_active.data
        db.session.commit()
        return redirect(url_for("business_dashboard"))

    return render_template("product_form.html", form=form, product=product)

```

Business order detail

```
@app.route("/business/orders/<int:order_id>")
@login_required
def business_order_detail(order_id):
    producer = current_user.producer_profile
    # TODO: restrict to orders containing this producer's products
    order = Order.query.get_or_404(order_id)
    return render_template("business_order_detail.html", order=order)
```

Now your template links work:

- `url_for('edit_producer_profile')` → `/business/profile/edit`
- `url_for('edit_producer_address')` → `/business/address/edit`
- `url_for('create_product')` → `/business/products/new`
- `url_for('edit_product', product_id=product.id)` → `/business/products/<id>/edit`
- `url_for('business_order_detail', order_id=order.id)` → `/business/orders/<id>` [\[227\]](#) [\[228\]](#)

✱

apparently i also need ProductForm, EditAddressForm, EditProducerProfileForm

Exactly, you now need three WTForms classes in `forms.py` (or split into multiple files later):
ProductForm, EditAddressForm, EditProducerProfileForm. [\[229\]](#) [\[230\]](#)

Below are minimal versions that match the routes and your models.

ProductForm

```
from flask_wtf import FlaskForm
from wtforms import StringField, TextAreaField, DecimalField, BooleanField, SubmitField
from wtforms.validators import InputRequired, Length, NumberRange

class ProductForm(FlaskForm):
    name = StringField(
        "Produktname",
        validators=[InputRequired(), Length(min=2, max=255)],
    )
    description = TextAreaField(
        "Beschreibung",
        validators=[Length(max=2000)],
    )
    price = DecimalField(
        "Preis (€)",
        places=2,
        rounding=None,
        validators=[InputRequired(), NumberRange(min=0)],
    )
    is_active = BooleanField("Aktiv anzeigen")
```

```
submit = SubmitField("Speichern")
```

Use this form for both `create_product` and `edit_product` (obj=product für Edit). [\[231\]](#) [\[232\]](#)

EditAddressForm

```
from wtforms import StringField, SubmitField
from wtforms.validators import InputRequired, Length, Regexp

class EditAddressForm(FlaskForm):
    street = StringField(
        "Straße und Hausnummer",
        validators=[InputRequired(), Length(min=3, max=255)],
    )
    zip_code = StringField(
        "PLZ",
        validators=[
            InputRequired(),
            Length(min=4, max=10),
            Regexp(r"^[0-9]+$", message="Bitte nur Ziffern eingeben."),
        ],
    )
    city = StringField(
        "Stadt",
        validators=[InputRequired(), Length(min=2, max=100)],
    )
    country = StringField(
        "Land",
        validators=[InputRequired(), Length(min=2, max=100)],
    )

    submit = SubmitField("Adresse speichern")
```

This matches your Address model (zip_code in form → zip im Model).

EditProducerProfileForm

```
from wtforms import StringField, SubmitField
from wtforms.validators import InputRequired, Length, Email

class EditProducerProfileForm(FlaskForm):
    display_name = StringField(
        "Öffentlicher Name",
        validators=[InputRequired(), Length(min=2, max=255)],
    )
    legal_name = StringField(
        "Rechtlicher Name (optional)",
        validators=[Length(max=255)],
    )
    contact_email = StringField(
        "Kontakt-E-Mail",
        validators=[InputRequired(), Email(), Length(max=255)],
    )
```

```

    )
    contact_phone = StringField(
        "Telefon (optional)",
        validators=[Length(max=50)],
    )

    submit = SubmitField("Profil speichern")

```

Using them in the routes

- In `edit_producer_profile`:

```
form = EditProducerProfileForm(obj=producer)
```

- In `edit_producer_address`:

```
form = EditAddressForm(obj=producer.address)
```

- In `create_product / edit_product`:

```
form = ProductForm(obj=product) # only for edit
```

The `obj=` pattern pre-fills the form fields from the SQLAlchemy object, and after `validate_on_submit()` you copy values back to the model before committing. [\[233\]](#) [\[231\]](#)



now adress is relevant for both users, add a edit adress button which shows the fields to edit the adress in bouth my account and business account page

Use **dieselbe EditAddressForm** und ein gemeinsames Edit-Address-Route-Pattern, aber rufe es von beiden Dashboards aus auf. [\[244\]](#) [\[245\]](#)

1. Route: Adresse bearbeiten (einmal zentral)

Passe deine bisherige `edit_producer_address`-Route so an, dass sie für beide Kontotypen funktioniert:

```

@app.route("/address/edit", methods=["GET", "POST"])
@login_required
def edit_address():
    # Kundenadresse oder Producer-Adresse holen
    if current_user.role == "PRODUCER":
        producer = current_user.producer_profile
        address = producer.address

```

```

else:
    # falls du eine direkte User-Address-Relation hast, sonst anpassen
    address = current_user.address

form = EditAddressForm(obj=address)

if form.validate_on_submit():
    address.street = form.street.data
    address.zip = form.zip_code.data
    address.city = form.city.data
    address.country = form.country.data
    db.session.commit()

    # Nach Rollentyp zurückleiten
    if current_user.role == "PRODUCER":
        return redirect(url_for("business_dashboard"))
    else:
        return redirect(url_for("account"))

return render_template("edit_address.html", form=form)

```

Damit brauchst du **nur eine** Formular-Klasse `EditAddressForm` und **nur eine** Route. [\[246\]](#) [\[247\]](#)

2. Button im Kundenkonto (`account.html`)

Ersetze in deinem User-Dashboard:

```

<section class="account-section">
  <h2>Standard-Lieferadresse</h2>
  <p>{{ user.address }}</p>
  <p>{{ user.zip_code }} {{ user.city }}</p>
  <a href="{{ url_for('account') }}">Adresse bearbeiten</a>
</section>

```

durch:

```

<section class="account-section">
  <h2>Standard-Lieferadresse</h2>
  <p>{{ user.address }}</p>
  <p>{{ user.zip_code }} {{ user.city }}</p>
  <a href="{{ url_for('edit_address') }}">Adresse bearbeiten</a>
</section>

```

3. Button im Business-Dashboard (`business_dashboard.html`)

Im Business-Dashboard-Template:

```

<section class="account-section">
  <h2>Betriebsadresse</h2>
  {% if producer.address %}
    <p>{{ producer.address.street }}</p>
    <p>{{ producer.address.zip }} {{ producer.address.city }}</p>
  {% else %}
    <p>Keine Adresse angegeben</p>
  {% endif %}

```



```

        <p>{{ producer.address.country }}</p>
    {% else %}
        <p>Noch keine Adresse hinterlegt.</p>
    {% endif %}
    <a href="{{ url_for('edit_address') }}">Adresse bearbeiten</a>
</section>

```

Beide Buttons zeigen auf dieselbe Route `edit_address`, die anhand von `current_user.role` entscheidet, welche Adresse geladen und wohin zurückgeleitet wird. [\[248\]](#) [\[249\]](#)

✱✱

address = current_user.address why does this fail in this block

```

@app.route("/address/edit", methods=["GET", "POST"])
@login_required
def edit_address():
    # Kundenadresse oder Producer-Adresse holen
    if current_user.role == "PRODUCER":
        producer = current_user.producer_profile
        address = producer.address
    else:
        # falls du eine direkte User-Address-Relation hast, sonst anpassen
        address = current_user.address

```

```

form = EditAddressForm(obj=address)

if form.validate_on_submit():
    address.street = form.street.data
    address.zip = form.zip_code.data
    address.city = form.city.data
    address.country = form.country.data
    db.session.commit()

    # Nach Rollentyp zurückleiten
    if current_user.role == "PRODUCER":
        return redirect(url_for("business_dashboard"))
    else:
        return redirect(url_for("account"))

return render_template("edit_address.html", form=form)

```

`current_user.address` schlägt fehl, weil dein User-Model keine direkte address-Beziehung oder -Spalte hat. [\[260\]](#)

Warum es kracht

Für Kunden hast du aktuell nur so etwas wie:

- `ProducerProfile.address` → Beziehung zu Address
- Aber **kein** `User.address` oder `user.address_id` Feld

Dann liefert `current_user.address` einen `AttributeError: 'User' object has no attribute 'address'`.^[260]

Zwei saubere Optionen

Option A: Adresse immer über ProducerProfile

Wenn nur Produzenten eine Adresse haben sollen:

```
if current_user.role == "PRODUCER":
    producer = current_user.producer_profile
    address = producer.address
else:
    address = None # oder Redirect / Fehlermeldung
```

Dann brauchst du für „normalen“ User-Account eine andere Lösung oder ein eigenes Address-Konzept.

Option B (empfohlen): User bekommt eigene Address-Beziehung

Modell anpassen:

```
class User(UserMixin, db.Model):
    # ...
    address_id = db.Column(db.Integer, db.ForeignKey("address.address_id"), nullable=True)
    address = db.relationship("Address")
```

Beim Signup des Kunden setzt du dann `user.address_id` auf eine Address.^[260]

Die Edit-Route wird dann:

```
@app.route("/address/edit", methods=["GET", "POST"])
@login_required
def edit_address():
    if current_user.role == "PRODUCER":
        address = current_user.producer_profile.address
    else:
        address = current_user.address

    if not address:
        # ggf. neue Address anlegen
        address = Address(street="", zip="", city="", country="")
        db.session.add(address)
```

```

db.session.flush()
if current_user.role == "PRODUCER":
    current_user.producer_profile.address = address
else:
    current_user.address = address
db.session.commit()

form = EditAddressForm(obj=address)

if form.validate_on_submit():
    address.street = form.street.data
    address.zip = form.zip_code.data
    address.city = form.city.data
    address.country = form.country.data
    db.session.commit()
    return redirect(url_for(
        "business_dashboard" if current_user.role == "PRODUCER" else "account"
    ))

return render_template("edit_address.html", form=form)

```

Damit funktioniert derselbe Button „Adresse bearbeiten“ in **Kundenkonto** und **Business-Dashboard**, und `current_user.address` ist nur dann erlaubt, wenn du die Beziehung tatsächlich im User-Model definiert hast. [\[261\]](#) [\[262\]](#)

✱

eine adresse haben customerprofile und producerprofile aber nicht der user

Für Kunden musst du dann nicht `current_user.address`, sondern das **CustomerProfile** verwenden, genau wie bei Produzenten das **ProducerProfile**. [\[275\]](#)

1. Adresslogik an Profile koppeln

Wenn deine Modelle so ähnlich aussehen:

```

class CustomerProfile(db.Model):
    customer_id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("user.id"), unique=True, nullable=False)
    address_id = db.Column(db.Integer, db.ForeignKey("address.address_id"))
    address = db.relationship("Address", back_populates="customers")
    user = db.relationship("User", back_populates="customer_profile")

class ProducerProfile(db.Model):
    producer_id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("user.id"), unique=True, nullable=False)
    address_id = db.Column(db.Integer, db.ForeignKey("address.address_id"))
    address = db.relationship("Address", back_populates="producers")
    user = db.relationship("User", back_populates="producer_profile")

```

dann gibt es **keine** address direkt am User, sondern nur:

- `current_user.customer_profile.address`
- `current_user.producer_profile.address`

2. Gemeinsame Edit-Adresse-Route für beide Profile

Passe die Route so an:

```
@app.route("/address/edit", methods=["GET", "POST"])
@login_required
def edit_address():
    # 1) Adresse je nach Rolle holen
    if current_user.role == "PRODUCER":
        profile = current_user.producer_profile
    else:
        profile = current_user.customer_profile

    address = profile.address

    # Falls noch keine Adresse existiert, neue anlegen und verknüpfen
    if address is None:
        address = Address(street="", zip="", city="", country="")
        db.session.add(address)
        db.session.flush() # erzeugt address.address_id
        profile.address = address
        db.session.commit()

    # 2) Formular mit bestehender Adresse befüllen
    form = EditAddressForm(obj=address)

    # 3) POST: speichern
    if form.validate_on_submit():
        address.street = form.street.data
        address.zip = form.zip_code.data
        address.city = form.city.data
        address.country = form.country.data
        db.session.commit()

    # 4) Zur richtigen Seite zurück
    if current_user.role == "PRODUCER":
        return redirect(url_for("business_dashboard"))
    else:
        return redirect(url_for("account"))

    return render_template("edit_address.html", form=form)
```

Hier greifst du **bewusst** über das passende Profil auf die Adresse zu und nicht über `current_user.address`, das es im User-Model gar nicht gibt. [\[276\]](#) [\[277\]](#)

class CustomerProfile(db.Model):

```
__tablename__ = "customer_profile"

customer_id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True)
address_id = db.Column(db.Integer, db.ForeignKey("address.address_id"), nullable=True)
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

user = db.relationship("User", back_populates="customer_profile")
address = db.relationship("Address", back_populates="customers")
```

class ProducerProfile(db.Model):

tablename = "producer_profile"

```
producer_id = db.Column(db.Integer, primary_key=True)
user_id = db.Column(db.Integer, db.ForeignKey("user.id"), nullable=False, unique=True)
address_id = db.Column(db.Integer, db.ForeignKey("address.address_id"), nullable=True)
display_name = db.Column(db.String(255), nullable=False)
legal_name = db.Column(db.String(255), nullable=True)
tax_id = db.Column(db.String(50), nullable=True)
contact_email = db.Column(db.String(255), nullable=False)
contact_phone = db.Column(db.String(50), nullable=True)
verification_status = db.Column(db.String(20), nullable=False, default="pending")
created_at = db.Column(db.DateTime, default=datetime.utcnow, nullable=False)

user = db.relationship("User", back_populates="producer_profile")
products = db.relationship("Product", back_populates="producer", lazy="dynamic")
address = db.relationship("Address", back_populates="producers")
```

mein model sieht so aus

Genau, damit **dürfen** `current_user.customer_profile.address` und `current_user.producer_profile.address` verwendet werden – aber **nicht** `current_user.address`.
[\[290\]](#)

Passende edit_address-Route zu deinem Model

Mit deinen Profil-Modellen sollte die Route so aussehen:

```
@app.route("/address/edit", methods=["GET", "POST"])
@login_required
def edit_address():
    # 1) Profil je nach Rolle holen
    if current_user.role == "PRODUCER":
        profile = current_user.producer_profile
    else:
        profile = current_user.customer_profile
```

```

address = profile.address # kommt aus address_id + relationship

# Falls noch keine Adresse existiert: neue anlegen und verknüpfen
if address is None:
    address = Address(street="", zip="", city="", country="")
    db.session.add(address)
    db.session.flush() # erzeugt address.address_id
    profile.address = address # setzt profile.address_id
    db.session.commit()

# 2) Formular mit bestehender Adresse befüllen
form = EditAddressForm(obj=address)

# 3) POST: Änderungen speichern
if form.validate_on_submit():
    address.street = form.street.data
    address.zip = form.zip_code.data
    address.city = form.city.data
    address.country = form.country.data
    db.session.commit()

# 4) Zur passenden Seite zurück
if current_user.role == "PRODUCER":
    return redirect(url_for("business_dashboard"))
else:
    return redirect(url_for("account"))

return render_template("edit_address.html", form=form)

```

Wichtig:

- **Nie** `current_user.address` verwenden, weil User keine Address-Relation hat.
- Immer über das jeweilige Profil gehen:
 - Kunde: `current_user.customer_profile.address`
 - Produzent: `current_user.producer_profile.address`

Damit funktionieren der Edit-Address-Button im **Kundenkonto** und im **Business-Dashboard** mit derselben Route und demselben Formular. [\[291\]](#) [\[292\]](#)

✱

AttributeError

AttributeError: 'NoneType' object has no attribute 'address'

at this line: `address = profile.address # kommt aus address_id + relationship`

`profile` ist in diesem Moment `None`, also kannst du nicht `.address` darauf aufrufen. [\[293\]](#)

Warum `profile` `None` ist

In der Route:

```
if current_user.role == "PRODUCER":
    profile = current_user.producer_profile
else:
    profile = current_user.customer_profile
address = profile.address
```

tritt der Fehler auf, wenn:

- der eingeloggte User zwar `role == "CUSTOMER"` oder `"PRODUCER"` hat,
- aber **kein** passender Datensatz in `customer_profile` bzw. `producer_profile` existiert.

Dann liefert SQLAlchemy für `current_user.customer_profile` eben `None`.^[294]

Sicherer Fix für die Route

Baue eine Prüfung und ggf. Redirect/Fehler ein:

```
@app.route("/address/edit", methods=["GET", "POST"])
@login_required
def edit_address():
    # Profil je nach Rolle holen
    if current_user.role == "PRODUCER":
        profile = current_user.producer_profile
        back_endpoint = "business_dashboard"
    else:
        profile = current_user.customer_profile
        back_endpoint = "account"

    # Falls kein Profil existiert → sauber behandeln
    if profile is None:
        # z.B. zurück zum Account / Business mit Hinweis
        flash("Für dieses Konto existiert noch kein Profil.", "warning")
        return redirect(url_for(back_endpoint))

    address = profile.address

    # Falls noch keine Adresse existiert: neu anlegen und verknüpfen
    if address is None:
        address = Address(street="", zip="", city="", country="")
        db.session.add(address)
        db.session.flush()
        profile.address = address
        db.session.commit()

    form = EditAddressForm(obj=address)

    if form.validate_on_submit():
        address.street = form.street.data
        address.zip = form.zip_code.data
        address.city = form.city.data
```

```

        address.country = form.country.data
        db.session.commit()
        return redirect(url_for(back_endpoint))

    return render_template("edit_address.html", form=form)

```

Wichtig ist außerdem:

- Beim Signup des Kunden wirklich ein `CustomerProfile` anlegen und mit `user_id` verknüpfen.
- Beim Producer-Signup analog ein `ProducerProfile` anlegen (hast du bereits). [\[295\]](#)

✱

def edit_address():

```

if current_user.role == "PRODUCER":
    profile = current_user.producer_profile
    back_endpoint = "business_dashboard"
else:
    profile = current_user.customer_profile
    back_endpoint = "account"

if profile is None:
    print("No profile found for user")
    return redirect(url_for(back_endpoint))

```

why does profile is None execute

`profile is None` wird ausgeführt, weil für den eingeloggten User in der Datenbank **kein passender Profil-Datensatz** existiert – also kein `CustomerProfile` bzw. `ProducerProfile` mit `user_id = current_user.id`. [\[310\]](#) [\[311\]](#)

Typische Ursachen in deinem Setup

Eine oder mehrere der folgenden Situationen trifft zu:

- Beim **Customer-Signup** legst du nur einen User an, aber **kein** `CustomerProfile`, z.B.:

```

user = User(...)
db.session.add(user)
db.session.commit()
# Kein CustomerProfile(...)

```

Dann ist `current_user.customer_profile` später `None`.

- Beim **Producer-Signup** legst du zwar ein `ProducerProfile` an, aber du testest die Route gerade mit einem **Customer-Account** (oder umgekehrt), sodass für diese Rolle kein Profil existiert.

- Es gibt alte User in der DB (von früheren Tests), die kein Profil bekommen haben, du loggst dich mit so einem Account ein und rufst `/address/edit` auf.

So kannst du es schnell prüfen

In einer Route oder der Shell:

```
print(current_user.id, current_user.role)
print(current_user.customer_profile)
print(current_user.producer_profile)
```

Wenn eines davon `None` ist, gibt es in der jeweiligen Tabelle keinen Datensatz mit dieser `user_id`.
[\[312\]](#)

Was du jetzt konkret tun musst

1. Beim Customer-Signup ein Profil anlegen (falls noch nicht):

```
# nach dem Anlegen von user
customer_profile = CustomerProfile(user_id=user.id)
db.session.add(customer_profile)
db.session.commit()
```

2. Sicherstellen, dass du `/address/edit` mit einem User testest, der:

- `role == "CUSTOMER"` und ein `CustomerProfile` hat, oder
- `role == "PRODUCER"` und ein `ProducerProfile` hat.

3. Alte Test-User ohne Profil entweder löschen oder ihnen per Shell ein Profil nachträglich geben:

```
u = User.query.get(1)
cp = CustomerProfile(user_id=u.id)
db.session.add(cp)
db.session.commit()
```

Solange zu einem User kein `CustomerProfile/ProducerProfile` existiert, bleibt `profile` in deiner Route `None` und der `if profile is None:-Block` wird ausgeführt. [\[313\]](#) [\[314\]](#)

✱✱

ich möchte das immer ein user zuerst erstellt wird, dann je nachdem über welche page (login oder business login) and profile dazu erstellt wird (customer oder producer)

Du kannst genau so arbeiten: **immer zuerst** User, später – abhängig vom Einstieg (Kunden-Login vs. Business-Login) – das passende Profil (CustomerProfile oder ProducerProfile) anlegen. ^[315]
_[316]

1. Idee: Lazy-Profile nach Login/Signup anlegen

- Registrierung / erster Login über **Kunden-Flow** → `User.role = "CUSTOMER"` und bei Bedarf CustomerProfile anlegen.
- Registrierung / erster Login über **Business-Flow** → `User.role = "PRODUCER"` und ProducerProfile anlegen.
- Solange kein Profil existiert, ist `current_user.customer_profile` bzw. `.producer_profile` eben None – das musst du in deinen Routen abfangen und dann das Profil neu erzeugen. ^[317]

Beispiel: im Customer-Flow

Wenn du eine `/account`-Route hast:

```
@app.route("/account")
@login_required
def account():
    # Falls noch kein CustomerProfile existiert, jetzt anlegen
    if current_user.customer_profile is None:
        customer_profile = CustomerProfile(user_id=current_user.id)
        db.session.add(customer_profile)
        db.session.commit()
    return render_template("account.html", user=current_user)
```

Beispiel: im Business-Flow

In deiner `/business-Signup`-Route machst du es ja schon: dort erzeugst du nach dem User auch ein ProducerProfile.

Falls du später einen Producer-Account „aus einem bestehenden Kunden“ machen willst, kannst du bei Bedarf ein ProducerProfile nachziehen, sobald er den Business-Flow betritt.

2. edit_address passend zu dieser Logik

Mit „User zuerst, Profil später“ brauchst du in `edit_address` zwei Checks:

1. Existiert ein Profil für diese Rolle? Wenn nicht, anlegen.
2. Existiert eine Address für dieses Profil? Wenn nicht, anlegen.

```

@app.route("/address/edit", methods=["GET", "POST"])
@login_required
def edit_address():
    # 1) Profil je nach Rolle holen oder anlegen
    if current_user.role == "PRODUCER":
        profile = current_user.producer_profile
        back_endpoint = "business_dashboard"
        if profile is None:
            profile = ProducerProfile(
                user_id=current_user.id,
                display_name=current_user.first_name or "Mein Betrieb",
                contact_email=current_user.email,
            )
            db.session.add(profile)
            db.session.flush()
        else:
            profile = current_user.customer_profile
            back_endpoint = "account"
            if profile is None:
                profile = CustomerProfile(user_id=current_user.id)
                db.session.add(profile)
                db.session.flush()

    # 2) Adresse holen oder anlegen
    address = profile.address
    if address is None:
        address = Address(street="", zip="", city="", country="")
        db.session.add(address)
        db.session.flush()
        profile.address = address
        db.session.commit()

    # 3) Formular
    form = EditAddressForm(obj=address)

    if form.validate_on_submit():
        address.street = form.street.data
        address.zip = form.zip_code.data
        address.city = form.city.data
        address.country = form.country.data
        db.session.commit()
        return redirect(url_for(back_endpoint))

    return render_template("edit_address.html", form=form)

```

Damit passt die Logik zu deinem Wunsch:

- Ein User kann existieren ohne Profile.
- Je nach Flow wird das passende Profil automatisch erstellt, sobald es benötigt wird.
- Die gleiche /address/edit-Route funktioniert für Kunden und Produzenten. [\[315\]](#) [\[317\]](#)

sollte nicht beim signup wenn ein user erstellt wird auch direkt ein customerprofile oder producerprofile erstellt werden?

Ja, das ist in deinem Fall die sauberste und einfachste Lösung: Beim Signup direkt **User + passendes Profil** erstellen, statt das Profil später „on demand“ zu erzeugen. [\[330\]](#) [\[331\]](#)

Warum direkt Profil anlegen?

- Dein Code kann sich darauf verlassen, dass zu jedem User **immer** ein passendes Profil existiert.
- `current_user.customer_profile / current_user.producer_profile` sind dann nie `None` → weniger If-Branches und Fehler.
- Die `edit_address`-Route wird deutlich simpler, weil du kein Profil mehr nachträglich erzeugen musst. [\[332\]](#)

Konkrete Empfehlung für dich

- **Kunden-Signup** (normale Registrierung):
 - User mit `role="CUSTOMER"` anlegen.
 - Direkt danach ein `CustomerProfile(user_id=user.id)` erstellen und committen.
- **Business-Signup** (dein /business-Flow):
 - User mit `role="PRODUCER"` anlegen.
 - Direkt danach ein `ProducerProfile(user_id=user.id, display_name=..., contact_email=...)` erstellen und committen.

Dann kannst du in `edit_address` einfach:

```
if current_user.role == "PRODUCER":
    profile = current_user.producer_profile
    back_endpoint = "business_dashboard"
else:
    profile = current_user.customer_profile
    back_endpoint = "account"

address = profile.address
# Falls noch keine Adresse: nur Address anlegen, nicht mehr Profil
```

Wenn du möchtest, kannst du im nächsten Schritt kurz deinen Customer-Signup-Code zeigen, dann lässt sich dort konkret einbauen, wo `CustomerProfile` angelegt werden soll.



if step == "signup":

adjust the signup form, make it create a customerprofile attacht to the user

```
form = SignupForm()
```

```
if form.validate_on_submit():
```

```
first_name = form.first_name.data
```

```
last_name = form.last_name.data
```

```
username = form.username.data
```

```
password = form.password.data
```

```
        user = User(
            email=email,
            password_hash=generate_password_hash(password),
            first_name=first_name,
            last_name=last_name,
            role="CUSTOMER"
        )
        db.session.add(user)
        db.session.commit()
        login_user(user)
        return redirect(url_for("home"))
    return render_template("login.html", step="signup", email=email, form=form)
else: # treat anything else as signup
    form = SignupForm()
    return render_template("login.html", step="signup", email=email, form=form)
```

Beim Signup solltest du direkt nach dem User auch ein CustomerProfile mit `user_id=user.id` anlegen. [\[333\]](#) [\[334\]](#)

Angepasster Signup-Block mit CustomerProfile

```
if step == "signup":
    form = SignupForm()
    if form.validate_on_submit():
        first_name = form.first_name.data
        last_name = form.last_name.data
        username = form.username.data # falls du ihn brauchst
        password = form.password.data

        # 1) User anlegen
        user = User(
            email=email,
            password_hash=generate_password_hash(password),
            first_name=first_name,
            last_name=last_name,
            role="CUSTOMER",
        )
        db.session.add(user)
        db.session.flush() # sorgt dafür, dass user.id gesetzt ist

        # 2) CustomerProfile anlegen und mit User verknüpfen
        customer_profile = CustomerProfile(
```

```

        user_id=user.id,
        # address_id bleibt erstmal None, wird später über edit_address gesetzt
    )
    db.session.add(customer_profile)

    # 3) Alles zusammen committen
    db.session.commit()

    login_user(user)
    return redirect(url_for("home"))

return render_template("login.html", step="signup", email=email, form=form)
else:
    form = SignupForm()
    return render_template("login.html", step="signup", email=email, form=form)

```

Jetzt existiert für jeden neu registrierten Kunden automatisch ein `CustomerProfile`, sodass `current_user.customer_profile` in `account` und `edit_address` nicht mehr `None` ist. [\[335\]](#) [\[336\]](#)



und jetzt für business signup

Beim Business-Signup machst du das Gleiche wie beim Customer-Signup: direkt nach dem User auch ein `ProducerProfile` erstellen und verknüpfen. [\[337\]](#) [\[338\]](#)

Angenommen, dein Business-Flow ist weiter in `/business` unter `step == "signup"`:

```

if step == "signup":
    form = SignupFormProducers()
    if form.validate_on_submit():
        # 1) User anlegen
        user = User(
            first_name=form.first_name.data,
            last_name=form.last_name.data,
            email=form.email.data.strip().lower(),
            role="PRODUCER",
        )
        user.set_password(form.password.data)
        db.session.add(user)
        db.session.flush() # user.id ist jetzt gesetzt

        # 2) Address für den Betrieb anlegen
        address = Address(
            street=form.street.data,
            zip=form.zip_code.data,
            city=form.city.data,
            country=form.country.data,
        )
        db.session.add(address)
        db.session.flush() # address.address_id ist jetzt gesetzt

        # 3) ProducerProfile anlegen und mit User + Address verknüpfen

```

```

    producer_profile = ProducerProfile(
        user_id=user.id,
        address_id=address.address_id,
        display_name=form.display_name.data,
        legal_name=form.legal_name.data or None,
        tax_id=form.tax_id.data or None,
        contact_email=form.contact_email.data.strip().lower(),
        contact_phone=form.contact_phone.data or None,
        # verification_status bleibt default="pending"
    )
    db.session.add(producer_profile)

    # 4) Alles zusammen speichern
    db.session.commit()

    login_user(user)
    return redirect(url_for("business_dashboard"))

return render_template("business.html", step="signup", email=email, form=form)

```

Damit gilt jetzt:

- Jeder Customer-Signup erzeugt: User (role="CUSTOMER") + CustomerProfile.
- Jeder Business-Signup erzeugt: User (role="PRODUCER") + ProducerProfile + Address.

Und in allen späteren Routen kannst du sicher mit `current_user.customer_profile` bzw. `current_user.producer_profile` und deren address arbeiten, ohne None-Checks auf Profil-Ebene. [\[339\]](#) [\[340\]](#)

**

create a basic edit_address.html where i can show the form and the input field for the address changes. this is the form:

```

class EditAddressForm(FlaskForm):
    street = StringField(
        "Straße und Hausnummer",
        validators=[InputRequired(), Length(min=3, max=255)],
    )
    zip_code = StringField(
        "PLZ",
        validators=[
            InputRequired(),
            Length(min=4, max=10),
            Regexp(r"\[354\]+$", message="Bitte nur Ziffern eingeben."),
        ],
    )
    city = StringField(

```

```

"Stadt",
validators=[InputRequired(), Length(min=2, max=100)],
)
country = StringField(
"Land",
validators=[InputRequired(), Length(min=2, max=100)],
)

```

```
submit = SubmitField("Adresse speichern")
```

Eine einfache `edit_address.html` kann das Formular mit CSRF-Token, Inputs und Fehlermeldungen anzeigen. [\[355\]](#) [\[356\]](#)

```

{% extends "base.html" %}
{% block title %}Adresse bearbeiten{% endblock %}

{% block content %}
<h1>Adresse bearbeiten</h1>

<form method="post" action="{{ url_for('edit_address') }}">
  {{ form.csrf_token }}

  <div class="form-group">
    {{ form.street.label }}
    {{ form.street(class="form-control") }}
    {% for error in form.street.errors %}
      <div class="error">{{ error }}</div>
    {% endfor %}
  </div>

  <div class="form-group">
    {{ form.zip_code.label }}
    {{ form.zip_code(class="form-control") }}
    {% for error in form.zip_code.errors %}
      <div class="error">{{ error }}</div>
    {% endfor %}
  </div>

  <div class="form-group">
    {{ form.city.label }}
    {{ form.city(class="form-control") }}
    {% for error in form.city.errors %}
      <div class="error">{{ error }}</div>
    {% endfor %}
  </div>

  <div class="form-group">
    {{ form.country.label }}
    {{ form.country(class="form-control") }}
    {% for error in form.country.errors %}
      <div class="error">{{ error }}</div>
    {% endfor %}
  </div>

```



```

    <button type="submit" class="btn btn-primary">
      {{ form.submit.label.text }}
    </button>
  </form>
{% endblock %}

```

Wenn du Bootstrap nutzt, kannst du die `form-group`, `form-control` und `btn btn-primary` Klassen direkt übernehmen. [\[357\]](#) [\[358\]](#)

**

actually i think i pop up interface is more user friendly since we want to stay in the my account page and not open up a whole new page only for the edits

Ja, ein Popup (Modal) auf der „Mein Konto“-Seite ist dafür perfekt. Unten ein einfaches Beispiel mit Bootstrap-Modal, das dein `EditAddressForm` verwendet. [\[370\]](#) [\[371\]](#)

Button in account/business Template

In `account.html` und `business_dashboard.html` kannst du denselben Button nutzen:

```

<section class="account-section">
  <h2>Adresse</h2>
  ...
  <button type="button"
    class="btn btn-secondary"
    data-bs-toggle="modal"
    data-bs-target="#editAddressModal">
    Adresse bearbeiten
  </button>
</section>

```

Modal mit Formular (z.B. in `base.html` oder direkt im Account-Template)

```

{{# Bootstrap 5 Modal #}}
<div class="modal fade" id="editAddressModal" tabindex="-1" aria-labelledby="editAddressLabel">
  <div class="modal-dialog">
    <div class="modal-content">
      <form method="post" action="{{ url_for('edit_address') }}">
        <div class="modal-header">
          <h5 class="modal-title" id="editAddressLabel">Adresse bearbeiten</h5>
          <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
        </div>

        <div class="modal-body">
          {{ form.csrf_token }}

```

```

<div class="mb-3">
    {{ form.street.label(class="form-label") }}
    {{ form.street(class="form-control") }}
    {% for error in form.street.errors %}
        <div class="text-danger small">{{ error }}</div>
    {% endfor %}
</div>

<div class="mb-3">
    {{ form.zip_code.label(class="form-label") }}
    {{ form.zip_code(class="form-control") }}
    {% for error in form.zip_code.errors %}
        <div class="text-danger small">{{ error }}</div>
    {% endfor %}
</div>

<div class="mb-3">
    {{ form.city.label(class="form-label") }}
    {{ form.city(class="form-control") }}
    {% for error in form.city.errors %}
        <div class="text-danger small">{{ error }}</div>
    {% endfor %}
</div>

<div class="mb-3">
    {{ form.country.label(class="form-label") }}
    {{ form.country(class="form-control") }}
    {% for error in form.country.errors %}
        <div class="text-danger small">{{ error }}</div>
    {% endfor %}
</div>
</div>

<div class="modal-footer">
    <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Abbrechen
    <button type="submit" class="btn btn-primary">
        {{ form.submit.label.text }}
    </button>
</div>
</form>
</div>
</div>
</div>

```

Wichtig:

- Die Route `edit_address` bleibt wie gebaut (POST verarbeitet, danach redirect zurück zu `account` oder `business_dashboard`). [\[372\]](#) [\[373\]](#)
- In der jeweiligen View musst du `form = EditAddressForm(obj=address)` an das Template mitgeben, in dem der Modal steht, damit die Felder vorbefüllt sind.

```
{% block extra_head %}{% endblock %}
```

FoodLoop

[Startseite](#) [Produkte](#) [Produzenten](#) [Karte](#) [Warenkorb](#) {% if current_user.is_authenticated %} [Mein Konto](#) {% else %} [Login](#) {% endif %} [Join as a Business](#) {% block content %}{% endblock %}

FoodLoop – Regionale Lebensmittel ohne Umwege. © {{ 2025 }}

where does this code go

Das ist dein `base.html` Layout-Template und gehört nach `templates/base.html`. [\[385\]](#) [\[386\]](#)

Wo du es verwendest

- Datei anlegen: `your_project/templates/base.html` mit genau diesem Inhalt.
- Alle anderen Seiten erben davon, z.B.:

```
{# account.html #}
{% extends "base.html" %}
{% block title %}Mein Konto – FoodLoop{% endblock %}

{% block content %}
    <!-- dein Account-Inhalt -->
{% endblock %}
```

Dann bekommst du überall automatisch:

- Kopfbereich mit Logo und Navigation
- `<head>` mit Title-Block, CSS und optional `extra_head`
- Footer mit deinem FoodLoop-Text



{# Bootstrap 5 Modal #}

...

Adresse bearbeiten

... ..

```
{{ form.csrf_token }}
{{ form.street.label(class="form-label") }} {{ form.street(class="form-control") }} {% for error in
form.street.errors %} ``
{{ error }}
`` {% endfor %}
{{ form.zip_code.label(class="form-label") }} {{ form.zip_code(class="form-control") }} {% for
error in form.zip_code.errors %} ``
{{ error }}
`` {% endfor %}
```

```

{{ form.city.label(class="form-label") }} {{ form.city(class="form-control") }} {% for error in
form.city.errors %} ``
{{ error }}
`` {% endfor %}
{{ form.country.label(class="form-label") }} {{ form.country(class="form-control") }} {% for error
in form.country.errors %} ``
{{ error }}
`` {% endfor %}
`` Abbrechen `` {{ form.submit.label.text }}

```

wo füge ich das ein

Der Modal-Block gehört in das Template, in dem du auch den „Adresse bearbeiten“-Button und das `EditAddressForm`-Objekt zur Verfügung hast, also z.B. in `account.html` (und analog ins Business-Dashboard). [\[387\]](#) [\[388\]](#)

1. In welches Template?

- **Kundenkonto:** `templates/account.html`
- **Business:** `templates/business_dashboard.html`

Dort hast du bereits:

```

{% extends "base.html" %}
{% block content %}
...
<button type="button"
        class="btn btn-secondary"
        data-bs-toggle="modal"
        data-bs-target="#editAddressModal">
    Adresse bearbeiten
</button>
...
{% endblock %}

```

Direkt **unterhalb** dieses Buttons (aber noch innerhalb von `{% block content %}`) fügst du den Modal-Code ein:

```

{% block content %}
...
<button ...>Adresse bearbeiten</button>

{# HIER den Modal einfügen #}
<div class="modal fade" id="editAddressModal" ...>
...
</div>
{% endblock %}

```

Wichtig: In der zugehörigen Route musst du `form = EditAddressForm(obj=address)` setzen und mit `render_template("account.html", form=form, ...)` in das Template geben.

2. Bootstrap einbinden

Damit der Modal überhaupt funktioniert, brauchst du in `base.html` Bootstrap-CSS und -JS,

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel=
...
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
```

Dann klappt der Modal auf, ohne die Seite zu verlassen, und sendet das Formular an deine `edit_address-Route`.[\[389\]](#) [\[390\]](#)

✱

```
{% block extra_head %}{% endblock %}
```

FoodLoop

[Startseite](#) [Produkte](#) [Produzenten](#) [Karte](#) [Warenkorb](#) {% if current_user.is_authenticated %} [Mein Konto](#) {% else %} [Login](#) {% endif %} [Join as a Business](#) {% block content %}{% endblock %}

FoodLoop – Regionale Lebensmittel ohne Umwege. © {{ 2025 }}

this is base, where do i paste the modal connection

In `base.html` brauchst du nur **Bootstrap einbinden**, den eigentlichen Modal-HTML-Block lässt du im jeweiligen Seiten-Template (z.B. `account.html`).[\[403\]](#) [\[404\]](#)

1. Bootstrap in base.html einfügen

```
<!DOCTYPE html>
<html lang="de">
<head>
  <meta charset="UTF-8">
  <title>{% block title %}FoodLoop{% endblock %}</title>
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"

  <link rel="stylesheet" href="{{ url_for('static', filename='css/styles.css') }}">
  {% block extra_head %}{% endblock %}
</head>
<body>
<header>
  ...
</header>

<main>
  {% block content %}{% endblock %}
</main>

<footer>
  FoodLoop – Regionale Lebensmittel ohne Umwege. &copy; {{ 2025 }}
```

```

</footer>

<!-- Bootstrap JS (für Modal) -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js">
</body>
</html>

```

2. Modal-HTML in der Seite (z.B. account.html)

In account.html:

```

{% extends "base.html" %}
{% block content %}
...
<button type="button"
        class="btn btn-secondary"
        data-bs-toggle="modal"
        data-bs-target="#editAddressModal">
    Adresse bearbeiten
</button>

    {%# HIER den gesamten Modal-Code einfügen #}
    <div class="modal fade" id="editAddressModal" ...>
        ...
    </div>
{% endblock %}

```

Gleiches Muster im Business-Dashboard. Base kümmert sich um CSS/JS, die **Modal-Struktur** gehört in das Template, wo du den Button und das `form`-Objekt hast. [\[405\]](#) [\[406\]](#)

✱✱

{% extends "base.html" %}

```
{% block title %}Account{% endblock %}
```

```
{% block content %}
```

Mein Konto

Eingeloggt als: {{ session["user_email"] }}

Persönliche Daten

Name: {{ user.first_name }} {{ user.last_name }}

...

E-Mail: {{ user.email }}

...

Konto angelegt am: {{ user.created_at.strftime("%d.%m.%Y") }}

...

Standard-Lieferadresse

...

{{ user.address }}

{{ user.zip_code }} {{ user.city }}

Adresse bearbeiten

Adresse bearbeiten

{# HIER den Modal einfügen #}

...

Adresse bearbeiten

... ..

{% endblock %}

Bestellungen

{% if orders %}

 {% for order in orders %}

- `` **Bestellung #{{ order.id }}** – `` {{ order.created_at.strftime("%d.%m.%Y") }} – Gesamt:
 {{ order.total }} € `` Details ``

 {% endfor %}

{% else %}

Du hast noch keine Bestellungen.

{% endif %} Logout

{% endblock %}

was fehlt hier für das editaddress pop up

Es fehlen zwei Dinge:

1. Das Formular selbst im Modal ({{ form... }}),
2. Die Struktur deines Templates – du hast {% endblock %} zu früh gesetzt, der Bestell-Teil liegt außerhalb des Blocks. ^[407] ^[408]

Korrigierte account.html mit Address-Modal

```
{% extends "base.html" %}
{% block title %}Account{% endblock %}

{% block content %}
    <h1>Mein Konto</h1>

    <p>Eingeloggt als: {{ session["user_email"] }}</p>

    <section class="account-section">
        <h2>Persönliche Daten</h2>
        <p><strong>Name:</strong> {{ user.first_name }} {{ user.last_name }}</p>
        <p><strong>E-Mail:</strong> {{ user.email }}</p>
        <p><strong>Konto angelegt am:</strong> {{ user.created_at.strftime("%d.%m.%Y") }}</p>
    </section>

    <section class="account-section">
        <h2>Standard-Lieferadresse</h2>
        {% if address %}
            <p>{{ address.street }}</p>
            <p>{{ address.zip }} {{ address.city }}</p>
            <p>{{ address.country }}</p>
        {% else %}
            <p>Noch keine Adresse hinterlegt.</p>
        {% endif %}

        <button type="button"
            class="btn btn-primary"
            data-bs-toggle="modal"
            data-bs-target="#editAddressModal">
            Adresse bearbeiten
        </button>
    </section>

    {# Modal mit Formular #}
    <div class="modal fade" id="editAddressModal" tabindex="-1" aria-labelledby="editAddressModalLabel">
        <div class="modal-dialog">
            <div class="modal-content">
                <form method="post" action="{{ url_for('edit_address') }}">
                    <div class="modal-header">
                        <h5 class="modal-title" id="editAddressModalLabel">Adresse bearbeiten</h5>
                        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
                    </div>
                    <div class="modal-body">
                        {{ form.csrf_token }}

                        <div class="mb-3">
                            {{ form.street.label(class="form-label") }}
                            {{ form.street(class="form-control") }}
                            {% for error in form.street.errors %}
                                <div class="text-danger small">{{ error }}</div>
                            {% endfor %}
                        </div>
                    </div>
                </form>
            </div>
        </div>
    </div>
```



```

<div class="mb-3">
  {{ form.zip_code.label(class="form-label") }}
  {{ form.zip_code(class="form-control") }}
  {% for error in form.zip_code.errors %}
    <div class="text-danger small">{{ error }}</div>
  {% endfor %}
</div>

<div class="mb-3">
  {{ form.city.label(class="form-label") }}
  {{ form.city(class="form-control") }}
  {% for error in form.city.errors %}
    <div class="text-danger small">{{ error }}</div>
  {% endfor %}
</div>

<div class="mb-3">
  {{ form.country.label(class="form-label") }}
  {{ form.country(class="form-control") }}
  {% for error in form.country.errors %}
    <div class="text-danger small">{{ error }}</div>
  {% endfor %}
</div>
</div>

<div class="modal-footer">
  <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">Abbrechen
  <button type="submit" class="btn btn-primary">
    {{ form.submit.label.text }}
  </button>
</div>
</form>
</div>
</div>
</div>

<section class="account-section">
  <h2>Bestellungen</h2>
  {% if orders %}
    <ul>
      {% for order in orders %}
        <li>
          <strong>Bestellung #{{ order.id }}</strong> -
          {{ order.created_at.strftime("%d.%m.%Y") }} -
          Gesamt: {{ order.total }} €
          <a href="{{ url_for('order_detail', order_id=order.id) }}">Details</a>
        </li>
      {% endfor %}
    </ul>
  {% else %}
    <p>Du hast noch keine Bestellungen.</p>
  {% endif %}
</section>

<form action="{{ url_for('logout') }}" method="post">
  <button type="submit">Logout</button>

```

```
</form>
{% endblock %}
```

Was du in der Route übergeben musst

In der account-Route:

```
@app.route("/account")
@login_required
def account():
    user = current_user
    profile = user.customer_profile
    address = profile.address if profile else None
    form = EditAddressForm(obj=address)
    return render_template("account.html", user=user, address=address, orders=orders, fo
```

Dann hat das Template:

- user, address, orders
- form für das Modal

und das Popup funktioniert auf der gleichen Seite. [\[409\]](#) [\[407\]](#)

✱

@app.route("/account")

```
@login_required
def account():
    user = current_user
    profile = user.customer_profile
    address = profile.address if profile else None
    form = EditAddressForm(obj=address)
    if current_user.customer_profile:
        customer_profile = current_user.customer_profile.orders.all()
    else:
        customer_profile = []
    return render_template("account.html", user=user, orders=customer_profile, address=address,
form=form)
```

check this code

Die Logik ist fast richtig, aber zwei Dinge solltest du ändern: den Namen von customer_profile (du überschreibst den Profil-Begriff mit einer Order-Liste) und sicherstellen, dass orders über eine Relationship am CustomerProfile kommen. [\[410\]](#) [\[411\]](#)

Verbesserte Version der Route

Angenommen, dein `CustomerProfile` hat eine Relationship `orders` (z.B. `orders = db.relationship("Order", back_populates="customer_profile", lazy="dynamic")`), dann sieht die Route besser so aus:

```
@app.route("/account")
@login_required
def account():
    user = current_user
    profile = user.customer_profile
    address = profile.address if profile else None

    # Formular für Adress-Modal, mit vorhandener Adresse vorbefüllt
    form = EditAddressForm(obj=address)

    # Bestellungen des Kunden holen
    if profile and hasattr(profile, "orders"):
        orders = profile.orders.all() # Query -> Liste
    else:
        orders = []

    return render_template(
        "account.html",
        user=user,
        orders=orders,
        address=address,
        form=form,
    )
```

Wichtig dabei:

- `profile` bleibt das **Profil-Objekt**, `orders` ist die Liste der Bestellungen.
- In `account.html` iterierst du über `orders`, nicht über `customer_profile`.
- Falls du noch keine `orders`-Relationship im Modell hast, musst du sie im `CustomerProfile/Order`-Modell ergänzen, damit `.orders.all()` funktioniert. [\[411\]](#) [\[412\]](#)

✱✱

then i dont need this route anymore, im not using a seperate html to chagne the existing profile address:

```
@app.route("/address/edit", methods=["GET", "POST"])
@login_required
def edit_address():
    if current_user.role == "PRODUCER":
        profile = current_user.producer_profile
        back_endpoint = "business_dashboard"
```

else:

profile = current_user.customer_profile

back_endpoint = "account"

```
if profile is None:
    print("No profile found for user")
    return redirect(url_for(back_endpoint))

address = profile.address

# falls adresse nicht existiert, erstelle eine
if address is None:
    address = Address(street="", zip="", city="", country="")
    db.session.add(address)
    db.session.flush()          # erzeugt address.address_id
    profile.address = address   # setzt profile.address_id
    db.session.commit()

#wenn adresse besteht, aber geandert werden mochte
form = EditAddressForm(obj=profile.address)
if form.validate_on_submit():
    address.street = form.street.data
    address.zip = form.zip_code.data
    address.city = form.city.data
    address.country = form.country.data
    db.session.commit()

# Nach Rollentyp zurückleiten
if current_user.role == "PRODUCER":
    return redirect(url_for("business_dashboard"))
else:
    return redirect(url_for("account"))

return render_template("edit_address.html", form=form)
```

Die Route **brauchst du weiterhin**, auch wenn du kein eigenes `edit_address.html` mehr nutzt. Sie verarbeitet den POST aus dem Modal und speichert die Adressänderungen. [\[425\]](#) [\[426\]](#)

Was sich ändert

- Früher: Route rendert `edit_address.html`.
- Jetzt: Route bekommt das Formular aus dem Modal (auf `/account` oder `business_dashboard`), speichert die Daten und leitet zurück.

Du passt nur das letzte `return` an:

```
@app.route("/address/edit", methods=["GET", "POST"])
@login_required
def edit_address():
```

```

if current_user.role == "PRODUCER":
    profile = current_user.producer_profile
    back_endpoint = "business_dashboard"
else:
    profile = current_user.customer_profile
    back_endpoint = "account"

if profile is None:
    print("No profile found for user")
    return redirect(url_for(back_endpoint))

address = profile.address

# falls Adresse nicht existiert, erstelle eine
if address is None:
    address = Address(street="", zip="", city="", country="")
    db.session.add(address)
    db.session.flush()
    profile.address = address
    db.session.commit()

form = EditAddressForm(obj=address)

if form.validate_on_submit():
    address.street = form.street.data
    address.zip = form.zip_code.data
    address.city = form.city.data
    address.country = form.country.data
    db.session.commit()
    return redirect(url_for(back_endpoint))

# Neu: zurück auf die Seite, die das Modal anzeigt
# Hier kein eigenes Template mehr
return redirect(url_for(back_endpoint))

```

Im account-View renderst du `account.html` mit `form = EditAddressForm(obj=address)`, im Template steckt das Modal mit `action="{{ url_for('edit_address') }}"`.

Kurz:

- **Route behalten** (für POST + Datenlogik).
- Kein separates `edit_address.html` mehr, stattdessen Modal im `account.html/business_dashboard.html`. [\[427\]](#) [\[425\]](#)

✱✱

<section class="account-section">

```

...
<h2>Standard-Lieferadresse</h2>
...
<p>{{ user.address }}</p>

```

```
<p>{{ user.zip_code }} {{ user.city }}</p>
<a href="{{ url_for('edit_address') }}">Adresse bearbeiten</a>
```

</section>

Adresse bearbeiten

In diesem Abschnitt greifst du noch auf Felder zu, die es so gar nicht am User gibt und du hast den „Adresse bearbeiten“-Button doppelt. [\[428\]](#) [\[429\]](#)

Was du ändern solltest

1. Adresse vom CustomerProfile verwenden

In deiner account-Route holst du bereits:

```
profile = user.customer_profile
address = profile.address if profile else None
```

Im Template sollte es dann so aussehen:

```
<section class="account-section">
  <h2>Standard-Lieferadresse</h2>
  {% if address %}
    <p>{{ address.street }}</p>
    <p>{{ address.zip }} {{ address.city }}</p>
    <p>{{ address.country }}</p>
  {% else %}
    <p>Noch keine Adresse hinterlegt.</p>
  {% endif %}

  <button type="button"
    class="btn btn-primary"
    data-bs-toggle="modal"
    data-bs-target="#editAddressModal">
    Adresse bearbeiten
  </button>
</section>
```

2. Den alten Link entfernen

Diese Zeile kann weg, weil du jetzt das Modal verwendest:

```
<a href="{{ url_for('edit_address') }}">Adresse bearbeiten</a>
```

Die Adressänderung läuft nun ausschließlich über den Button + Modal, der POST geht an edit_address, und danach leitest du zurück auf /account. [\[430\]](#) [\[431\]](#)

i want the edit address button to look like the first button, but the first button doesnt work, so delete the first button and change the apperance of the second button

Entferne den ersten Link und behalte nur den Modal-Button. Hier die bereinigte Version:

```
<section class="account-section">
  <h2>Standard-Lieferadresse</h2>
  {% if address %}
    <p>{{ address.street }}</p>
    <p>{{ address.zip }} {{ address.city }}</p>
    <p>{{ address.country }}</p>
  {% else %}
    <p>Noch keine Adresse hinterlegt.</p>
  {% endif %}

  <button type="button"
    class="btn btn-link"
    data-bs-toggle="modal"
    data-bs-target="#editAddressModal">
    Adresse bearbeiten
  </button>
</section>
```

Was geändert wurde:

1. ✗ Ersten Link `<a href="{{ url_for('edit_address') }}">` gelöscht
2. ✓ Button behalten, aber `class="btn btn-primary"` → `class="btn btn-link"` geändert

`btn-link` macht den Button optisch wie einen normalen Link (blau, unterstrichen beim Hover, kein Hintergrund). Falls du einen anderen Style möchtest (z.B. `btn-secondary`, `btn-outline-primary`), sag Bescheid.

make the save button functional

Der Save-Button sollte bereits funktionieren, wenn das Setup korrekt ist. Prüfe folgende Punkte:

1. Route `edit_address` muss POST verarbeiten

Deine Route sollte so aussehen (du hast sie schon, aber zur Sicherheit):

```
@app.route("/address/edit", methods=["GET", "POST"])
@login_required
def edit_address():
    if current_user.role == "PRODUCER":
        profile = current_user.producer_profile
        back_endpoint = "business_dashboard"
    else:
```

```

        profile = current_user.customer_profile
        back_endpoint = "account"

    if profile is None:
        return redirect(url_for(back_endpoint))

    address = profile.address

    if address is None:
        address = Address(street="", zip="", city="", country="")
        db.session.add(address)
        db.session.flush()
        profile.address = address
        db.session.commit()

    form = EditAddressForm() # WICHTIG: Ohne obj=, damit POST-Daten verwendet werden

    if form.validate_on_submit():
        address.street = form.street.data
        address.zip = form.zip_code.data
        address.city = form.city.data
        address.country = form.country.data
        db.session.commit()
        return redirect(url_for(back_endpoint))

    # Bei GET oder Validierungsfehler: Formular mit aktueller Adresse befüllen
    if request.method == "GET":
        form = EditAddressForm(obj=address)

    return redirect(url_for(back_endpoint))

```

Wichtig: EditAddressForm() ohne obj=address beim POST, sonst werden deine Eingaben ignoriert.

2. Account-Route muss Form mit Daten vorbefüllen

```

@app.route("/account")
@login_required
def account():
    user = current_user
    profile = user.customer_profile
    address = profile.address if profile else None

    form = EditAddressForm(obj=address) # Vorbefüllt für das Modal

    if profile and hasattr(profile, "orders"):
        orders = profile.orders.all()
    else:
        orders = []

    return render_template(
        "account.html",
        user=user,
        orders=orders,
        address=address,

```



```
        form=form,  
    )
```

3. Test

1. Öffne `/account`
2. Klicke „Adresse bearbeiten“ → Modal öffnet sich mit vorhandenen Daten
3. Ändere z.B. die Straße
4. Klicke „Adresse speichern“
5. → Seite lädt neu, Änderung ist sichtbar

Falls es nicht funktioniert, sag mir welchen Fehler du siehst (Browser-Konsole oder Flask-Log).

adresse speichern reagiert nicht

Wenn der „Adresse speichern“-Button nicht reagiert, liegt es meist an einem der folgenden Probleme:

Häufigste Ursachen & Fixes

1. Form-Action fehlt oder ist falsch

Das Modal-Formular muss explizit auf die Route zeigen:

```
<form method="post" action="{{ url_for('edit_address') }}">  
    {{ form.csrf_token }}  
    ...  
</form>
```

2. CSRF-Token fehlt

Stelle sicher, dass `{{ form.csrf_token }}` **innerhalb** des `<form>`-Tags steht, direkt nach dem öffnenden Tag.

3. Route akzeptiert keinen POST richtig

Überprüfe deine `edit_address`-Route – sie muss `methods=["GET", "POST"]` haben und das Formular korrekt validieren:

```
from flask import request  
  
@app.route("/address/edit", methods=["GET", "POST"])  
@login_required  
def edit_address():  
    if current_user.role == "PRODUCER":  
        profile = current_user.producer_profile  
        back_endpoint = "business_dashboard"
```

```

else:
    profile = current_user.customer_profile
    back_endpoint = "account"

if profile is None:
    return redirect(url_for(back_endpoint))

address = profile.address

if address is None:
    address = Address(street="", zip="", city="", country="")
    db.session.add(address)
    db.session.flush()
    profile.address = address
    db.session.commit()

form = EditAddressForm()

if request.method == "POST" and form.validate_on_submit():
    address.street = form.street.data
    address.zip = form.zip_code.data
    address.city = form.city.data
    address.country = form.country.data
    db.session.commit()
    return redirect(url_for(back_endpoint))

# GET: Formular vorbefüllen
form = EditAddressForm(obj=address)

# Für GET-Anfragen: zeige account-Seite mit vorhandenen Daten
return redirect(url_for(back_endpoint))

```

4. Debug: Prüfen, ob POST ankommt

Füge temporär ein `print()` in die Route ein:

```

if request.method == "POST":
    print("POST empfangen!")
    print(f"Form valid: {form.validate_on_submit()}")
    print(f"Form errors: {form.errors}")

```

Starte Flask neu, klicke „Adresse speichern“ und schau ins Terminal – wenn nichts erscheint, wird der POST gar nicht verarbeitet.

5. Button-Typ prüfen

Der Submit-Button **muss** `type="submit"` haben:

```

<button type="submit" class="btn btn-primary">
    Adresse speichern
</button>

```

Zeig mir den aktuellen Code vom Modal und der `edit_address`-Route, dann kann ich gezielter helfen.

1. <https://flask-login.readthedocs.io>
2. <https://flask-login.readthedocs.io/en/latest/>
3. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
4. <https://flask-login.readthedocs.io>
5. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
6. <https://flask-login.readthedocs.io/en/latest/>
7. <https://pypi.org/project/Flask-Login/>
8. <https://www.geeksforgeeks.org/python/extensions-in-flask/>
9. <https://pypi.org/project/Flask-Login/>
10. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
11. <https://stackoverflow.com/questions/58152229/sqlalchemy-exc-noreferencedcolumnerror-could-not-initialize-target-column-for-f>
12. <https://github.com/sqlalchemy/sqlalchemy/discussions/8236>
13. <https://dev.to/misbagas/sqlalchemy-foreign-key-error-could-not-find-table-user-for-announcementcreatorid-265f>
14. https://www.reddit.com/r/SQLAlchemy/comments/wqw9ep/sqlalchemy_sqlalchemyexcnoindexedtableerror/
15. <https://python-forum.io/thread-17949.html>
16. <https://github.com/miguelgrinberg/Flask-Migrate/issues/255>
17. <https://stackoverflow.com/questions/54774645/alembic-sqlalchemy-exc-noreferencedcolumnerror-using-flask-sqlalchemy-and-flas>
18. <https://github.com/sqlalchemy/sqlalchemy/issues/5350>
19. <http://docs-sqlalchemy.readthedocs.org/en/latest/core/exceptions.html>
20. <https://thoth-station.ninja/docs/developers/storages/modules/sqlalchemy/sql/schema.html>
21. https://www.youtube.com/watch?v=WAL3uDAF4_4
22. https://dokk.org/documentation/sqlalchemy/rel_1_2_19/core/exceptions/
23. <https://groups.google.com/g/sqlalchemy/c/hZ3NVf1S2cE>
24. <https://flask-login.readthedocs.io>
25. <https://flask-login.readthedocs.io/en/latest/>
26. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
27. <https://stackoverflow.com/questions/58152229/sqlalchemy-exc-noreferencedcolumnerror-could-not-initialize-target-column-for-f>
28. <https://flask-login.readthedocs.io>
29. <https://flask-login.readthedocs.io/en/latest/>

30. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
31. <https://stackoverflow.com/questions/58152229/sqlalchemy-exc-noreferencedcolumnerror-could-not-initialize-target-column-for-f>
32. <https://github.com/miguelgrinberg/Flask-Migrate/issues/255>
33. <https://stackoverflow.com/questions/54774645/alembic-sqlalchemy-exc-noreferencedcolumnerror-using-flask-sqlalchemy-and-flas>
34. <https://flask-login.readthedocs.io>
35. <https://flask-login.readthedocs.io/en/0.6.2/>
36. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
37. <https://stackoverflow.com/questions/78384234/flask-session-with-timeout-that-terminates-on-browser-close>
38. <https://testdriven.io/tips/b842d4dc-4d93-4d26-99fa-585e84395eaf/>
39. <https://flask-session.readthedocs.io/en/latest/config.html>
40. <https://www.youtube.com/watch?v=Tb4YygdLz24>
41. <https://www.youtube.com/watch?v=CRvV9nFKoPl>
42. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
43. <https://drdroid.io/framework-diagnosis-knowledge/python-flask-flask-login--remember-me-not-working>
44. <https://www.youtube.com/watch?v=C-h0VjjMzWA>
45. <https://github.com/pallets-eco/flask-session/issues/221>
46. <https://stackoverflow.com/questions/54422551/flask-login-remember-me-working-automatically>
47. https://www.reddit.com/r/flask/comments/d1u2r6/session_not_deleted_when_browser_closed/
48. <https://github.com/maxcountryman/flask-login/issues/269>
49. <https://stackoverflow.com/questions/58152229/sqlalchemy-exc-noreferencedcolumnerror-could-not-initialize-target-column-for-f>
50. <https://github.com/miguelgrinberg/Flask-Migrate/issues/255>
51. <https://stackoverflow.com/questions/54774645/alembic-sqlalchemy-exc-noreferencedcolumnerror-using-flask-sqlalchemy-and-flas>
52. <https://stackoverflow.com/questions/58152229/sqlalchemy-exc-noreferencedcolumnerror-could-not-initialize-target-column-for-f>
53. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
54. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
55. <https://github.com/miguelgrinberg/Flask-Migrate/issues/255>
56. <https://stackoverflow.com/questions/54774645/alembic-sqlalchemy-exc-noreferencedcolumnerror-using-flask-sqlalchemy-and-flas>
57. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
58. <https://flask-login.readthedocs.io>
59. <https://flask-login.readthedocs.io/en/latest/>

60. <https://flask-login.readthedocs.io/en/0.6.2/>
61. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
62. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
63. <https://github.com/miguelgrinberg/Flask-Migrate/issues/255>
64. <https://stackoverflow.com/questions/54774645/alembic-sqlalchemy-exc-noreferencedcolumnerror-using-flask-sqlalchemy-and-flas>
65. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
66. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
67. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
68. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
69. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
70. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
71. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
72. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
73. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
74. <https://sqlitebrowser.org>
75. <https://datacarpentry.github.io/sql-socialsci/02-db-browser.html>
76. <https://sqlitebrowser.org/dl/>
77. <https://dbschema.com/blog/sqlite/create-er-diagrams-sqlite/>
78. <https://dbmstools.com/categories/database-diagram-tools/sqlite>
79. <https://www.bytebase.com/blog/top-database-schema-diagram-tools/>
80. <https://github.com/sqlitebrowser/sqlitebrowser/wiki>
81. https://www.reddit.com/r/sqlite/comments/1ga0m4a/visualize_db_er_diagram_sqlite/
82. <https://www.heise.de/download/product/db-browser-for-sqlite-41685>
83. <https://www.youtube.com/watch?v=nwZLJiZ7-oQ>
84. <https://turso.tech/blog/5-best-free-sqlite-gui>
85. <https://github.com/sqlitebrowser/sqlitebrowser>
86. <https://www.youtube.com/watch?v=fPWZhVjvIU>
87. <https://news.ycombinator.com/item?id=40909076>
88. <https://www.youtube.com/watch?v=mhaY9hh3lrs>
89. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
90. <https://datacarpentry.github.io/sql-socialsci/02-db-browser.html>
91. <https://sqlitebrowser.org>

92. 0-9

93. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
94. <https://stackoverflow.com/questions/58152229/sqlalchemy-exc-noreferencedcolumnerror-could-not-initialize-target-column-for-f>
95. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
96. <https://flask-login.readthedocs.io>
97. <https://flask-login.readthedocs.io/en/0.6.2/>
98. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
99. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
100. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
101. <https://stackoverflow.com/questions/37227780/flask-session-persisting-after-close-browser>
102. <https://stackoverflow.com/questions/67999475/html-template-doesnt-display-passed-variables-flask>
103. <https://flask.palletsprojects.com/en/stable/templating/>
104. https://www.reddit.com/r/flask/comments/rfjrag/jinja_variable_not_showing_passed_through_render/
105. <https://github.com/pallets/jinja/issues/743>
106. <https://stackoverflow.com/questions/47952830/flask-login-current-user>
107. <https://teamtreehouse.com/community/html-render-template-not-working>
108. <https://jinja.palletsprojects.com/en/stable/templates/>
109. https://www.reddit.com/r/flask/comments/or17c6/how_do_my_templates_know_what_current_user_is/
110. <https://forum.djangoproject.com/t/raise-error-for-missing-variable-used-in-template/39776>
111. <https://www.pythonanywhere.com/forums/topic/14355/>
112. <https://stackoverflow.com/questions/61974693/flask-render-template-passed-variable-not-being-used-in-the-template>
113. https://www.reddit.com/r/flask/comments/znq9s0/my_template_is_not_rendering_yet_my_print/
114. https://runebook.dev/en/articles/flask/api/index/flask.render_template
115. <https://stackoverflow.com/questions/19614027/jinja2-template-variable-if-none-object-set-a-default-value>
116. <https://stackoverflow.com/questions/69859412/flask-flask-login-pytest-how-do-i-set-flask-logins-current-user>
117. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>
118. <https://stackoverflow.com/questions/58003926/flask-sqlalchemy-add-multiple-row>
119. <https://stackoverflow.com/questions/3659142/bulk-insert-with-sqlalchemy-orm>
120. https://www.reddit.com/r/dataengineering/comments/1exrr3s/bulk_insert_rows_using_sqlalchemy/
121. https://www.reddit.com/r/flask/comments/3tea4k/af_flasksqlalchemy_bulk_updateinsert/
122. <https://stackoverflow.com/questions/65641380/how-to-insert-multiple-rows-into-a-single-table-using-flask-sqlalchemy>
123. <https://github.com/sqlalchemy/sqlalchemy/discussions/7111>

124. <https://towardsdatascience.com/how-to-perform-bulk-inserts-with-sqlalchemy-efficiently-in-python-23044656b97d/>
125. <https://github.com/sqlalchemy/sqlalchemy/discussions/10537>
126. <https://www.geeksforgeeks.org/python/sqlalchemy-core-insert-multiple-rows-from-a-tuple-instead-of-dict/>
127. <https://www.youtube.com/watch?v=KPF9CRrdFJU>
128. <https://stackoverflow.com/questions/58517491/sqlalchemy-bulk-save-objects-vs-add-all-underlying-logic-difference/58517821>
129. <https://python-forum.io/thread-31907.html>
130. <https://www.geeksforgeeks.org/python/sqlalchemy-bulk-insert-with-one-to-one-relation/>
131. <https://flask.palletsprojects.com/en/stable/patterns/sqlalchemy/>
132. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>
133. <https://stackoverflow.com/questions/58003926/flask-sqlalchemy-add-multiple-row>
134. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>
135. <https://flask.palletsprojects.com/en/stable/patterns/sqlalchemy/>
136. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>
137. <https://flask.palletsprojects.com/en/stable/patterns/sqlalchemy/>
138. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>
139. <https://flask.palletsprojects.com/en/stable/templating/>
140. <https://flask.palletsprojects.com/en/stable/patterns/sqlalchemy/>
141. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>
142. <https://blog.getflits.com/9-features-on-a-customer-account-page-e-commerce/>
143. <https://crocoblock.com/blog/woocommerce-my-account-page-essentials/>
144. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
145. <https://www.justinmind.com/ui-design/dashboard-design-best-practices-ux>
146. <https://blog.getflits.com/9-features-on-a-customer-account-page-e-commerce/>
147. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>
148. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>
149. <https://flask-sqlalchemy.readthedocs.io/en/stable/queries/>
150. <https://flask.palletsprojects.com/en/stable/patterns/wtforms/>
151. <https://getdocs.org/Sqlalchemy/docs/latest/tutorial/metadata>
152. https://python-adv-web-apps.readthedocs.io/en/latest/flask_forms.html
153. <https://www.codepowered.com/manuals/SQLAlchemy-0.6.9-doc/html/core/schema.html>
154. <https://wtforms.readthedocs.io/page/validators/>
155. <https://krython.com/tutorial/python/flask-forms-wtforms/>
156. <https://stackoverflow.com/questions/9088957/sqlalchemy-cannot-find-a-class-name>
157. https://wtforms.readthedocs.io/en/2.3.x/_modules/wtforms/validators/
158. <https://stackoverflow.com/questions/48320834/accessing-sqlalchemy-relationship-data-from-jinja/48324783>

159. <https://pythonprogramming.net/flask-user-registration-form-tutorial/>
160. <https://www.youtube.com/watch?v=EpJRJsmqnn0>
161. <https://explore-flask.readthedocs.io/en/latest/forms.html>
162. <https://www.geeksforgeeks.org/python/flask-wtf-explained-how-to-use-it/>
163. <https://wtforms.readthedocs.io/en/2.3.x/validators/>
164. <https://www.youtube.com/watch?v=j5lQl4aW9ZU>
165. <https://stackoverflow.com/questions/23712986/pre-populate-a-wtforms-in-flask-with-data-from-a-sqlalchemy-object>
166. <https://dev.to/brunooliveira/flask-series-part-10-allowing-users-to-register-and-login-1enb>
167. https://python-adv-web-apps.readthedocs.io/en/latest/flask_forms.html
168. <https://stackoverflow.com/questions/30357643/is-it-possible-to-commit-two-related-tables-in-a-sqlalchemy-transaction>
169. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>
170. https://www.reddit.com/r/flask/comments/3exah5/should_i_validate_flaskwtfl_form_object_or/
171. <https://flask-user.readthedocs.io/en/v0.6/customization.html>
172. <https://www.youtube.com/watch?v=wwQJzMrKy9E>
173. <http://goonan.io/flask-wtf-tricks/>
174. <https://github.com/sqlalchemy/sqlalchemy/discussions/6901>
175. <https://pypi.org/project/WTForms-SQLAlchemy/>
176. <https://stackoverflow.com/questions/70555232/flask-how-to-access-current-user-when-creating-forms>
177. https://www.reddit.com/r/flask/comments/2om2fd/af_multiple_model_forms_w_wtforms_and_sqlalchemy/
178. <https://www.youtube.com/watch?v=d0jR-2UB9Y0>
179. <https://stackoverflow.com/questions/33767973/flask-populating-a-wtform-with-an-sqlalchemy-model-with-relationships>
180. <https://www.peterspython.com/en/blog/flask-sqlalchemy-crud-application-with-wtforms-queryselectfield-and-queryselectmultiplefield>
181. <https://flask.palletsprojects.com/en/stable/patterns/sqlalchemy/>
182. <https://flask-sqlalchemy.readthedocs.io/en/stable/quickstart/>
183. <https://stackoverflow.com/questions/4201455/sqlalchemy-whats-the-difference-between-flush-and-commit>
184. http://docs-sqlalchemy.readthedocs.org/ko/latest/orm/session_basics.html
185. <https://www.digitalocean.com/community/tutorials/how-to-use-flask-sqlalchemy-to-interact-with-databases-in-a-flask-application>
186. <https://deepnote.com/blog/ultimate-guide-to-sqlalchemy-library-in-python>
187. https://www.reddit.com/r/flask/comments/1c9l1n8/understanding_flasksqlalchemy_lifecycle/
188. <https://dev.to/brunooliveira/flask-series-part-10-allowing-users-to-register-and-login-1enb>
189. <https://mailtrap.io/blog/flask-email-verification/>
190. <https://stackoverflow.com/questions/44903485/does-db-session-commit-change-app-context-in-flask-sqlalchemy>

191. <https://roytuts.com/python-flask-multi-step-registration-form-with-mysql-jquery/>

192. https://github.com/FactoryBoy/factory_boy/issues/81

193. <https://michaelcho.me/article/sqlalchemy-commit-flush-expire-refresh-merge-whats-the-difference/>

194. <https://www.youtube.com/watch?v=-2lU10Og8U>

195. https://python-web.teclado.com/section13/lectures/02_registering_users_flask/

196. <https://fintechpython.pages.oit.duke.edu/jupyternotebooks/7-WebDevelopment/14-flask-template.html>

197. <https://pythonprogramming.net/flask-user-dashboard/>

198. <https://github.com/DiptoChakrabarty/flask-online-store>

199. https://www.youtube.com/watch?v=h4FFSlw_U6g

200. <https://www.youtube.com/watch?v=6hJFQaEyBt0>

201. <https://flask.palletsprojects.com/en/stable/patterns/wtforms/>

202. <https://stackoverflow.com/questions/46092054/flask-login-documentation-loginform>

203. <https://github.com/app-generator/jinja-template>

204. <https://pusher.com/tutorials/live-dashboard-python/>

205. <https://dev.to/sm0ke/soft-ui-dashboard-free-jinja-template-3b7k>

206. <https://github.com/app-generator/jinja-paper-dashboard>

207. <https://www.codementor.io/@chirilovadrian360/flask-dashboards-open-source-and-free-yzzi4c16h>

208. <https://stackoverflow.com/questions/24963077/flask-admin-template>

209. <https://www.youtube.com/watch?v=BdAciR-myGI>

210. <https://vibecodingwithfred.com/tutorials/shopping-flask/>

211. <https://stackoverflow.com/questions/31283690/undefinederror-url-for-other-page-is-undefined>

212. <https://flask.palletsprojects.com/en/stable/api/>

213. <http://explore-flask.readthedocs.org/en/latest/views.html>

214. <https://www.digitalocean.com/community/tutorials/how-to-use-templates-in-a-flask-application>

215. <https://www.youtube.com/watch?v=xIRxlqgmXpc>

216. <https://realpython.com/flask-blueprint/>

217. <https://flask.palletsprojects.com/en/stable/config/>

218. <https://www.youtube.com/watch?v=6hJFQaEyBt0>

219. <https://discuss.python.org/t/image-url-in-template/27535>

220. <https://dev.to/chukwunazaekpere/custom-routes-in-flask-using-flask-restful-4imj>

221. https://www.reddit.com/r/flask/comments/zmo21m/werkzeugroutingexceptionsbuildererror_could_not/

222. <https://github.com/flask-restful/flask-restful/issues/494>

223. <https://stackoverflow.com/questions/9023488/build-error-with-variables-and-url-for-in-flask>

224. <https://flask123.sinaapp.com/article/14>

225. <https://vibecodingwithfred.com/tutorials/shopping-flask/>

226. <https://www.digitalocean.com/community/tutorials/how-to-use-templates-in-a-flask-application>

227. <https://flask.palletsprojects.com/en/stable/api/>

228. <https://realpython.com/flask-blueprint/>

229. <https://flask.palletsprojects.com/en/stable/patterns/wtforms/>

230. <https://hackersandslackers.com/flask-wtforms-forms/>

231. <https://wtforms.readthedocs.io/en/3.0.x/forms/>

232. <https://wtforms.readthedocs.io/en/2.3.x/forms/>

233. <https://stackoverflow.com/questions/6196622/using-wtforms-populate-obj-method-with-flask-micro-framework>

234. <https://stackoverflow.com/questions/71728789/python-flask-email-address-to-the-right-of-a-flask-form-input-box>

235. <https://www.digitalocean.com/community/tutorials/how-to-use-and-validate-web-forms-with-flask-wtf>

236. <https://code.tutsplus.com/intro-to-flask-adding-a-contact-page--net-28982t>

237. <https://mailtrap.io/blog/flask-contact-form/>

238. https://python-adv-web-apps.readthedocs.io/en/latest/flask_forms.html

239. <https://flask.palletsprojects.com/en/stable/quickstart/>

240. <https://flask-wtf.readthedocs.io/en/0.15.x/form.html>

241. <https://github.com/hayleyking/examples-forms>

242. <https://stackoverflow.com/questions/27824110/flask-wtforms-dynamically-generated-list-of-fields>

243. <https://www.youtube.com/watch?v=j5lQl4aW9ZU>

244. <https://stackoverflow.com/questions/67283927/reuse-wtforms-field-in-multiple-forms>

245. <https://hackersandslackers.com/flask-wtforms-forms/>

246. https://python-adv-web-apps.readthedocs.io/en/latest/flask_forms.html

247. <https://www.digitalocean.com/community/tutorials/how-to-use-and-validate-web-forms-with-flask-wtf>

248. <https://www.youtube.com/watch?v=ikGWk8L3aWw>

249. <https://copyninja.in/blog/using-url-for-in-flask.html>

250. <https://www.youtube.com/watch?v=ip9TdHsxSJo>

251. <https://stackoverflow.com/questions/43404089/flask-admin-revise-the-button-text>

252. <https://stackoverflow.com/questions/29251004/rewrite-a-url-with-flask>

253. <https://teamtreehouse.com/community/multiple-forms-in-a-single-route-page-with-wtforms>

254. <https://flask-admin.readthedocs.io/en/stable/introduction/>

255. <https://flask.palletsprojects.com/en/stable/patterns/wtforms/>

256. <https://stackoverflow.com/questions/65066253/how-to-make-a-redirect-button-in-flask-template>

257. https://www.reddit.com/r/flask/comments/tbwlnm/multiple_forms_in_the_same_page_with_flaskwtforms/

258. <https://www.youtube.com/watch?v=zMXUIZvesil>

259. <https://dev.to/sampart/combining-multiple-forms-in-flask-wtforms-but-validating-independently-cbm>

260. <https://deepnote.com/blog/ultimate-guide-to-sqlalchemy-library-in-python>

261. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>

262. <https://www.youtube.com/watch?v=xRfupYowU-I>

263. <https://testdriven.io/blog/flask-social-auth/>

264. <https://community.plotly.com/t/problems-with-trying-to-implement-flask-login-into-my-multi-page-app/68599>
265. <https://stackoverflow.com/questions/9381536/sqlalchemy-one-to-one-relationship-on-the-same-table>
266. <https://flask-admin.readthedocs.io/en/stable/introduction/>
267. <https://discuss.python.org/t/add-custom-attributes-to-the-flask-login-current-user/13722>
268. <https://github.com/sqlalchemy/sqlalchemy/discussions/10943>
269. <https://stackoverflow.com/questions/12710036/attributeerror-on-current-user-is-authenticated>
270. <https://github.com/pallets-eco/flask-security/issues/518>
271. https://www.reddit.com/r/flask/comments/5hlcmj/af_flasksocketio_not_linking_to_current_user/
272. <https://github.com/maxcountryman/flask-login/issues/366>
273. <https://stackoverflow.com/questions/74654326/flask-admin-not-recognising-current-user-in-model-view>
274. <https://flask-security-too.readthedocs.io/en/stable/changelog.html>
275. <https://dev.to/freddiemazzilli/flask-sqlalchemy-relationships-exploring-relationship-associations-igo>
276. <https://stackoverflow.com/questions/61196511/how-to-correctly-define-relationships-in-a-python-sqlalchemy-class-with-multiple>
277. <https://stackoverflow.com/questions/3464443/how-to-create-one-to-one-relationships-with-declarative>
278. https://sqla-tutorials-nb.readthedocs.io/en/latest/orm_related_objects.html
279. <https://mailtrap.io/blog/flask-contact-form/>
280. <https://stackoverflow.com/questions/22364551/creating-flask-form-with-selects-from-more-than-one-table>
281. <https://www.digitalocean.com/community/tutorials/how-to-use-one-to-many-database-relationships-with-flask-sqlalchemy>
282. <https://github.com/lingthio/Flask-User/issues/192>
283. <https://github.com/sqlalchemy/sqlalchemy/discussions/10943>
284. <https://notes.kodekloud.com/docs/Python-API-Development-with-FastAPI/Advanced-FastAPI/Sqlalchemy-Relationships>
285. <https://www.linkedin.com/pulse/understanding-sqlalchemy-relationships-one-to-one-venkatesh-pardeshi-r5kpf>
286. https://www.reddit.com/r/SQLAlchemy/comments/1e0biyk/how_to_make_relationships_and_query_them/
287. <https://testdriven.io/blog/flask-sessions/>
288. <https://deepnote.com/blog/ultimate-guide-to-sqlalchemy-library-in-python>
289. <https://flask-appbuilder.readthedocs.io/en/latest/relations.html>
290. <https://dev.to/freddiemazzilli/flask-sqlalchemy-relationships-exploring-relationship-associations-igo>
291. <https://stackoverflow.com/questions/61196511/how-to-correctly-define-relationships-in-a-python-sqlalchemy-class-with-multiple>
292. <https://stackoverflow.com/questions/3464443/how-to-create-one-to-one-relationships-with-declarative>
293. <https://www.thecodingforums.com/threads/query-from-sqlalchemy-returns-attributeerror-nonetype-object.960311/>

294. https://sqla-tutorials-nb.readthedocs.io/en/latest/orm_related_objects.html

295. <https://www.digitalocean.com/community/tutorials/how-to-use-one-to-many-database-relationships-with-flask-sqlalchemy>

296. <https://github.com/maxcountryman/flask-login/issues/135>

297. <https://github.com/dpgaspar/Flask-AppBuilder/issues/2046>

298. https://www.reddit.com/r/learnpython/comments/rkqegg/sqlalchemy_14x_error_attributeerror_nonetype/

299. <https://flask-login.readthedocs.io>

300. https://www.reddit.com/r/flask/comments/9s363q/flaskuser_user_has_no_attribute_get_user_by_token/

301. <https://www.pythonanywhere.com/forums/topic/33131/>

302. <https://flask-security-too.readthedocs.io/en/stable/configuration.html>

303. <https://github.com/sqlalchemy/sqlalchemy/issues/10098>

304. <https://stackoverflow.com/questions/63143352/flask-current-user-is-none-type>

305. <https://community.plotly.com/t/flask-login-in-plotly-dash-current-user-is-returning-none/71968>

306. <https://alexwlchan.net/til/2024/current-user-equals-none/>

307. <https://www.pythonanywhere.com/forums/topic/7488/>

308. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>

309. <https://stackoverflow.com/questions/2605205/nonetype-object-has-no-attribute-get-error-using-sqlalchemy>

310. https://sqla-tutorials-nb.readthedocs.io/en/latest/orm_related_objects.html

311. <https://www.thecodingforums.com/threads/query-from-sqlalchemy-returns-attributeerror-nonetype-object.960311/>

312. <https://deepnote.com/blog/ultimate-guide-to-sqlalchemy-library-in-python>

313. <https://stackoverflow.com/questions/3464443/how-to-create-one-to-one-relationships-with-declarative>

314. <https://www.digitalocean.com/community/tutorials/how-to-use-one-to-many-database-relationships-with-flask-sqlalchemy>

315. <https://stackoverflow.com/questions/75095594/modeling-a-one-to-one-relation-with-optional-tables-in-sqlalchemy>

316. <https://www.linkedin.com/pulse/understanding-sqlalchemy-relationships-one-to-one-venkatesh-pardeshi-r5kpf>

317. <https://hackersandslackers.com/sqlalchemy-data-models/>

318. https://www.reddit.com/r/flask/comments/82tskc/af_how_do_i_add_rolebased_authorization/

319. <https://dev.to/brunoliveira/flask-series-part-10-allowing-users-to-register-and-login-1enb>

320. <https://flask-appbuilder.readthedocs.io/en/v4.1.0/security.html>

321. <https://www.youtube.com/watch?v=Uzb9z5wNPPA>

322. <https://flask-user.readthedocs.io/en/v0.6/customization.html>

323. <https://stackoverflow.com/questions/15871391/implementing-flask-login-with-multiple-user-classes>

324. <https://stackoverflow.com/questions/55493848/flask-admin-set-up-default-user-roles-when-register>

325. <https://www.permit.io/blog/implement-role-based-access-control-in-flask>

326. <https://www.geeksforgeeks.org/python/flask-role-based-access-control/>

327. <https://www.youtube.com/watch?v=9v2NZ2dGB60>

328. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>

329. <https://www.pythonanywhere.com/forums/topic/7702/>

330. <https://stackoverflow.com/questions/75095594/modeling-a-one-to-one-relation-with-optional-tables-in-sqlalchemy>

331. <https://hackersandslackers.com/sqlalchemy-data-models/>

332. <https://www.linkedin.com/pulse/understanding-sqlalchemy-relationships-one-to-one-venkatesh-pardes-hi-r5kpf>

333. <https://stackoverflow.com/questions/75095594/modeling-a-one-to-one-relation-with-optional-tables-in-sqlalchemy>

334. <https://hackersandslackers.com/sqlalchemy-data-models/>

335. <https://www.linkedin.com/pulse/understanding-sqlalchemy-relationships-one-to-one-venkatesh-pardes-hi-r5kpf>

336. <https://www.digitalocean.com/community/tutorials/how-to-use-one-to-many-database-relationships-with-flask-sqlalchemy>

337. <https://stackoverflow.com/questions/20261276/flask-creating-a-unique-profile-page-for-each-user>

338. <https://coderpad.io/blog/development/understanding-transactions-in-sqlalchemy/>

339. <https://stackoverflow.com/questions/75095594/modeling-a-one-to-one-relation-with-optional-tables-in-sqlalchemy>

340. <https://hackersandslackers.com/sqlalchemy-data-models/>

341. <https://www.permit.io/blog/implement-role-based-access-control-in-flask>

342. <https://www.geeksforgeeks.org/python/how-to-add-authentication-to-your-app-with-flask-login/>

343. <https://stackoverflow.com/questions/4201455/sqlalchemy-whats-the-difference-between-flush-and-commit>

344. https://flask-user.readthedocs.io/en/latest/basic_app.html

345. https://www.reddit.com/r/learnpython/comments/8umdj5/flask_handling_multiple_users/

346. <https://mark.douthwaite.io/getting-production-ready-a-minimal-flask-app/>

347. <https://github.com/sqlalchemy/sqlalchemy/discussions/9566>

348. <https://flask-user.readthedocs.io/en/latest/authorization.html>

349. <https://dev.to/brunooliveira/flask-series-part-10-allowing-users-to-register-and-login-1enb>

350. <https://www.youtube.com/watch?v=kTJg-EWATdQ>

351. <https://fusionauth.io/blog/building-profile-portal-with-flask-oauth-apis>

352. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>

353. <https://michaelcho.me/article/sqlalchemy-commit-flush-expire-refresh-merge-whats-the-difference/>

354. 0-9

355. <https://flask-wtf.readthedocs.io/en/1.0.x/quickstart/>

356. <https://www.digitalocean.com/community/tutorials/how-to-use-and-validate-web-forms-with-flask-wtf>

357. <https://github.com/helloflask/bootstrap-flask>

358. <https://thepythoncode.com/article/front-end-of-crud-flask-app-using-jinja-and-bootstrap>

359. <https://stackoverflow.com/questions/78169468/how-to-generate-html5-nice-validation-error-messages-from-flask-wtforms-validation>

360. <https://flask-wtf.readthedocs.io/en/1.2.x/quickstart/>

361. <https://stackoverflow.com/questions/60715270/how-would-i-style-a-contact-form-with-jinja2-and-html>

362. https://www.reddit.com/r/flask/comments/bkbbid/multiple_forms_with_jinja_and_error_handling/

363. <https://fintechpython.pages.oit.duke.edu/jupyternotebooks/7-WebDevelopment/15-flask-forms.html>

364. <https://pythonprogramming.net/bootstrap-jinja-templates-flask/>

365. <https://stackoverflow.com/questions/16406496/rendering-forms-with-flask-wtform>

366. <https://stackoverflow.com/q/49442935>

367. https://www.tutorialspoint.com/flask/flask_wtf.htm

368. <https://www.youtube.com/watch?v=7df-ZY9q-2A>

369. <https://www.geeksforgeeks.org/python/flask-wtf-explained-how-to-use-it/>

370. <https://getbootstrap.com/docs/4.1/components/modal/>

371. <https://getbootstrap.com/docs/5.3/components/modal/>

372. <https://quaded.com/flask-bootstrap-modal-w-form-validation/>

373. <https://stackoverflow.com/questions/40446101/submit-wtf-flask-form-and-stay-on-same-page>

374. <https://dev.to/sampart/combining-multiple-forms-in-flask-wtforms-but-validating-independently-cbm>

375. <https://getbootstrap.com/docs/5.0/components/modal/>

376. <https://mdbootstrap.com/docs/standard/extended/modal-form/>

377. <https://codingyaar.com/bootstrap-5-modal-form/>

378. <https://htmx.org/examples/modal-bootstrap/>

379. https://www.reddit.com/r/flask/comments/tbwlnm/multiple_forms_in_the_same_page_with_flaskwtforms/

380. <https://stackoverflow.com/questions/61273612/implementing-a-modal-form-with-flask>

381. https://www.reddit.com/r/flask/comments/1il2wwh/i_am_using_flask_and_bootstrap_5_1_and_i_want_to/

382. <https://pypi.org/project/Flask-Modals/>

383. https://www.reddit.com/r/flask/comments/aadit3/how_to_stay_on_same_page_wo_redirectreload/

384. <https://www.youtube.com/watch?v=rKUKfDKQG6I>

385. <https://www.digitalocean.com/community/tutorials/how-to-use-templates-in-a-flask-application>

386. <https://flask.palletsprojects.com/en/stable/quickstart/>

387. <https://stackoverflow.com/questions/72019467/login-modal-content-from-another-page-with-flask-wtf-on-top-of-bootstrap-how-to>

388. <https://stackoverflow.com/questions/65535540/bootstrap-modal-not-appearing-jinja2>

389. <https://getbootstrap.com/docs/5.0/components/modal/>

390. <https://getbootstrap.com/docs/4.1/components/modal/>

391. <https://stackoverflow.com/questions/18290142/multiple-forms-in-a-single-page-using-flask-and-wtforms>

392. <https://ezzeddinabdullah.com/post/frontend-crud-flask-jinja-bootstrap/>

393. <https://getbootstrap.com/docs/4.0/components/modal/>

394. https://www.reddit.com/r/flask/comments/tbwlnm/multiple_forms_in_the_same_page_with_flaskwtforms/

395. <https://flask-admin.readthedocs.io/en/stable/introduction/>

396. <https://reactuatesoftware.com/flaskwtforms-create-and-customize-from-from-model/>

397. <https://github.com/pallets-eco/flask-admin/pull/2642>

398. https://www.reddit.com/r/flask/comments/oglowb/does_anyone_know_how_to_put_flaskuser/

399. <https://pypi.org/project/Flask-Modals/>

400. <https://quaded.com/flask-bootstrap-modal-w-form-validation/>

401. <https://testdriven.io/blog/developing-a-single-page-app-with-flask-and-vuejs/>

402. <https://www.newline.co/courses/fullstack-flask-course/styling-flask-jinja2-templates-with-bootstrap-css-and-if-logic>

403. <https://getbootstrap.com/docs/4.1/components/modal/>

404. <https://getbootstrap.com/docs/5.0/components/modal/>

405. <https://quaded.com/flask-bootstrap-modal-w-form-validation/>

406. <https://stackoverflow.com/questions/72019467/login-modal-content-from-another-page-with-flask-wtf-on-top-of-bootstrap-how-to>

407. <https://stackoverflow.com/questions/16406496/rendering-forms-with-flask-wtform>

408. <https://fintechpython.pages.oit.duke.edu/jupyternotebooks/7-WebDevelopment/15-flask-forms.html>

409. <https://www.digitalocean.com/community/tutorials/how-to-use-and-validate-web-forms-with-flask-wtf>

410. <https://www.digitalocean.com/community/tutorials/how-to-use-one-to-many-database-relationships-with-flask-sqlalchemy>

411. <https://www.linkedin.com/pulse/understanding-database-relationships-flask-sqlalchemy-adeniyi-5i38f>

412. <https://flask-sqlalchemy.readthedocs.io/en/stable/api/>

413. <https://stackoverflow.com/questions/46723767/how-to-get-current-user-when-implementing-python-flask-security>

414. <https://mailtrap.io/blog/flask-contact-form/>

415. <https://www.digitalocean.com/community/tutorials/how-to-use-many-to-many-database-relationships-with-flask-sqlalchemy>

416. <https://www.digitalocean.com/community/tutorials/how-to-add-authentication-to-your-app-with-flask-login>

417. <http://explore-flask.readthedocs.org/en/latest/forms.html>

418. <https://github.com/dpgaspar/Flask-AppBuilder/issues/2046>

419. <https://stackoverflow.com/questions/22308649/flask-sqlalchemy-order-by-relationship>

420. <https://github.com/sqlalchemy/sqlalchemy/discussions/10209>

421. <https://dev.to/freddiemazzilli/flask-sqlalchemy-relationships-exploring-relationship-associations-igo>

422. <https://www.youtube.com/watch?v=6hJFQaEyBt0>

423. <https://stackoverflow.com/questions/67097529/how-to-take-form-data-from-flask-template-and-pass-to-html-page>

424. <https://www.julo.ch/blog/sqlalchemy-many-to-many-ordering/>

- 425. <https://quaded.com/flask-bootstrap-modal-w-form-validation/>
- 426. <https://stackoverflow.com/questions/40446101/submit-wft-flask-form-and-stay-on-same-page>
- 427. <https://stackoverflow.com/questions/18290142/multiple-forms-in-a-single-page-using-flask-and-wtforms>
- 428. <https://dev.to/freddiemazzilli/flask-sqlalchemy-relationships-exploring-relationship-associations-igo>
- 429. <https://www.linkedin.com/pulse/understanding-database-relationships-flask-sqlalchemy-adeniyi-5i38f>
- 430. <https://quaded.com/flask-bootstrap-modal-w-form-validation/>
- 431. <https://stackoverflow.com/questions/40446101/submit-wft-flask-form-and-stay-on-same-page>